HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

CAPSTONE PROJECT REPORT

Retrieval-Augmented Generation for Legal Case Reasoning and Criminal Judgment Prediction

Submitted by: Nguyen Nhat Minh
Student ID: 20225510
Vu Duc Duy
Student ID: 202416686

Supervisor: Ph.D. Nguyen Kiem Hieu

Academic year: 2025 – 2026

HANOI, 2026

Abstract

While Large Language Models (LLMs) have driven substantial advancements in legal judgment prediction, their application in Vietnam remains limited by a scarcity of structured datasets and tailored reasoning frameworks. To address this gap, this project introduces a Retrieval-Augmented Generation (RAG) system integrated with a Reason-and-Act (ReAct) loop to accurately predict criminal charges and penalty durations. As a foundation, this research constructs a structured case law database by extracting and organizing data from criminal judgments sourced directly from the Supreme People's Court of Vietnam.

The proposed system is evaluated on a dataset comprising legal judgment PDFs collected from provincial first-instance criminal proceedings. Text extraction and structural formatting into JSON are achieved using the Marker Optical Character Recognition (OCR) model. The RAG architecture utilizes ChromaDB as the vector store and BGE-M3 for embeddings, while the ReAct loop is powered by the Gemma model via the Google AI Studio API. This technical combination enables the framework to synthesize retrieved legal facts with step-by-step logical deductions.

The primary contribution of this research is the establishment of a complete, end-to-end pipeline: transitioning from raw PDF processing to a structured legal retrieval system, and culminating in an LLM-driven predictive reasoning loop. By synergizing the direct retrieval of statutory articles with the context-aware retrieval of analogous historical cases, the system adeptly captures nuanced legal elements—particularly mitigating and aggravating circumstances. Identifying these factors is crucial for reliable and accurate judgment forecasting within the Vietnamese legal framework.

Contents

CHAPTER 1

Introduction

1.1 Motivation

Navigating legal case analysis and judgment prediction is highly complex, requiring the ability to interpret facts, apply relevant statutes, and weigh nuanced circumstances. While Large Language Models (LLMs) show immense potential for automating these tasks, relying solely on their pre-trained knowledge is risky. Without explicit context, LLMs are prone to hallucinations and outdated information, which is unacceptable in the legal domain where accuracy and strict traceability are essential.

Applying AI to the Vietnamese legal system presents additional hurdles. Not only are Vietnamese legal documents largely unstructured and published as raw PDFs, but criminal cases often hinge on subtle differences in intent and specific mitigating or aggravating factors. Accurately mapping case facts to the correct charges and penalty ranges requires a sophisticated framework that standard text generation models cannot provide.

To address these challenges, this project proposes a Retrieval-Augmented Generation (RAG) framework integrated with a multi-stage, retrieval-grounded reasoning pipeline for Vietnamese criminal judgment prediction. By dynamically retrieving relevant legal articles and historical precedents, the system grounds the LLM in factual data and guides it through a structured reasoning process. This approach significantly reduces hallucinations, ensures traceability, and highly improves the reliability of predicting criminal charges and penalty lengths.

While Large Language Models (LLMs) excel at complex reasoning, their application in Vietnamese legal judgement prediction remains underexplored compared to resource-rich languages. Furthermore, relying solely on standard LLMs risks information hallucination and often fails to capture the subtle factual nuances that differentiate similar criminal offences. To address these limitations, this project aims to develop a judgement prediction system for Vietnamese criminal law that replaces pure text generation with structured reasoning and external legal knowledge retrieval. The primary objectives of this project are:

1. To construct a structured dataset of Vietnamese criminal judgements for retrieval and evaluation.
2. To develop an integrated framework supporting legal article retrieval, similar case retrieval, and LLM-based reasoning.
3. To accurately predict criminal charges and penalty lengths based on extracted defendant information and case facts.
4. To evaluate the retrieval-augmented system against baseline LLM-only approaches.

The scope of this research is strictly limited to Vietnamese criminal law, specifically processing defendant profiles and case summaries to output charges and penalty ranges. It excludes

other legal branches such as civil, administrative, or commercial law. Finally, this system is a research prototype designed to test the efficacy of structured reasoning in legal AI. It is constrained by the quality of the collected data and is not intended to replace professional legal counsel or judicial decision-making

1.2 Proposed Method

To address the challenges previously outlined, this project proposes a Retrieval-Augmented Generation (RAG) framework paired with a multi-stage, retrieval-grounded reasoning pipeline for predicting Vietnamese criminal judgments. Rather than relying solely on an LLM's internal pre-trained knowledge, this system synthesizes three critical information sources: the input case facts, relevant statutory laws, and analogous historical judgments. By grounding the model in external legal databases, the system guides the AI through a structured, transparent reasoning process. First, to overcome the scarcity of structured Vietnamese legal data, this project constructs a comprehensive case law database derived from raw PDF judgments issued by the Supreme People's Court of Vietnam. Utilizing Optical Character Recognition (OCR), the system extracts text from these documents and formats it into structured JSON records. Key legal elements—such as defendant profiles, case summaries, applied statutes, charges, and penalties—are isolated and stored to facilitate subsequent retrieval and system evaluation. Second, to mitigate LLM hallucinations and ensure legal accuracy, the system utilizes a vector database to index legal articles and historical cases via an embedding model. During inference, the system retrieves statutory laws relevant to the case facts alongside past judgments with similar circumstances. This provides the LLM with verified legal evidence, enabling predictions based on actual statutes rather than speculative generation. Third, to manage the complexities of legal analysis, the project introduces a three-stage, retrieval-grounded reasoning pipeline. Inspired by the Reason-and-Act (ReAct) paradigm—but utilizing a predefined sequence of actions rather than dynamic LLM routing—this pipeline is structured as follows:

- **Stage 1: Fact Extraction and Candidate Proposal.** The LLM processes the defendant's information and case summary to extract structured factual elements (e.g., relevant conduct, prior convictions, mitigating/aggravating factors). Based on these facts, the model proposes multiple plausible candidate offenses under the Vietnamese Criminal Code, preventing premature commitment to a single charge.
- **Stage 2: Statutory Retrieval and Legal Analysis.** The system retrieves statutory texts corresponding to the candidate offenses, alongside a fixed set of supporting criminal laws (covering sentencing guidelines, complicity, recidivism, etc.). The LLM evaluates the extracted facts against these laws to select the most probable offense, determine the sentencing bracket, and justify the rejection of inapplicable candidates.
- **Stage 3: Precedent Retrieval and Final Prediction.** Finally, the system retrieves similar past cases matching the selected offense to serve as a baseline for sentencing calibration. Synthesizing the case facts, legal analysis, retrieved statutes, and precedent summaries, the LLM generates a comprehensive prediction detailing the final charge, applied legal clauses, and specific penalty durations.

Fourth, to differentiate between legally analogous violations, the system enforces an explicit comparison between case facts and the legal elements of candidate offenses. Because criminal

charges often hinge on subtle distinctions—such as intent or the specific role of a defendant—the LLM is required to explicitly justify its selected charge and explain its reasoning, thereby enhancing prediction reliability. Finally, system performance is evaluated against ground-truth judgments from the test dataset. Legal article prediction is measured using precision, recall, and F1 scores (normalized to the article level), while penalty predictions are evaluated by converting imprisonment terms into months and calculating the Root Mean Squared Error (RMSE). To validate the efficacy of structured reasoning, this proposed pipeline is benchmarked against standard LLM-only and basic retrieval baselines. In summary, this tentative solution delivers an end-to-end pipeline for Vietnamese criminal judgment prediction. By transforming raw PDFs into a structured database and deploying a meticulously staged LLM reasoning framework, this approach minimizes hallucination, enforces factual grounding, and ensures a transparent predictive process.

1.3 Contributions

1. **End-to-End Data Pipeline:** Developed a comprehensive pipeline to extract raw Vietnamese criminal judgement PDFs and convert them into highly structured JSON records. This process systematically categorizes essential legal elements, including defendant profiles, case facts, judicial reasoning, applied statutes, charges, and penalty outcomes.
2. **Dual-Retrieval Legal Database:** Constructed a specialized retrieval database indexing both Vietnamese statutory laws and historical criminal precedents. This infrastructure enables direct legal-article lookups and similar-case matching, which serve to factually ground the LLM's judgement predictions.
3. **Multi-Stage Reasoning Framework:** Proposed and implemented a retrieval-augmented reasoning pipeline tailored for Vietnamese criminal judgement prediction. This framework strategically decomposes the prediction task into sequential phases: factual extraction and candidate offense generation, statutory legal analysis, precedent retrieval, and the synthesis of a final structured verdict.

CHAPTER 2

Theoretical Background

This chapter presents the theoretical foundations of the main technologies used in this project. These concepts are necessary for understanding how the system processes raw legal judgement PDFs, converts them into structured data, builds a searchable vector database, and applies retrieval-grounded reasoning for Vietnamese criminal judgement prediction.

2.1 Embedding

Embeddings serve as continuous vector representations of textual data. Formally, an embedding model operates as a mapping function $f : \mathcal{X} \rightarrow \mathbb{R}^d$, where $\mathcal{X}$ represents the discrete linguistic input space (words, sentences, or documents) and d is the dimensionality of the continuous vector space. Within this geometric space, texts sharing semantic similarity are positioned in close proximity, whereas disparate texts are distanced from one another. This mathematical representation is the foundational mechanism driving semantic search, document retrieval, clustering, and numerous other natural language processing (NLP) applications.

Foundational embedding methodologies, such as Word2Vec [1] and GloVe [2], operated predominantly at the token level. These frameworks are predicated on the distributional hypothesis, generating a static embedding matrix where words sharing linguistic contexts exhibit high vector correlation. In the legal domain, for instance, terms like "court," "judge," and "defendant" frequently co-occur and are thus mapped to correlated regions in $\mathbb{R}^d$. The primary contribution of these early models was demonstrating that semantic meaning could be captured through learned weights rather than manually engineered features.

Despite this success, static word embeddings are inherently limited by their inability to resolve polysemy, as they assign a single, invariable vector $v_w \in \mathbb{R}^d$ to a given word w regardless of its surrounding context. For example, the term "sentence" denotes a grammatical construct in linguistics, yet signifies a penal sanction within jurisprudence. To overcome this limitation, contextualized embedding models such as ELMo [3] and BERT [4] were introduced. Rather than relying on fixed representations, these architectures dynamically compute a token's vector representation h_i as a function of the entire input sequence of length n:

$$h_i = f(w_1, w_2, \ldots, w_n)_i \tag{2.1}$$

where h_i encapsulates both the token and its specific contextual usage.

For information retrieval, systems typically require an aggregated representation of longer text spans, mapping an entire sequence to a single dense vector. Sentence-BERT (SBERT) [5] is a prominent solution that fine-tunes BERT-based architectures to produce fixed-size sentence embeddings. This enables highly efficient semantic comparison using distance metrics, most notably cosine similarity. Given a query vector $\mathbf{q}$ and a document vector $\mathbf{d}$, the semantic

similarity is computed as the normalized inner product:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2} \tag{2.2}$$

This optimization is crucial for large-scale semantic search operations, where a system must rapidly evaluate a query against a vast corpus of stored documents.

Within a Retrieval-Augmented Generation (RAG) architecture, embeddings act as the critical bridge connecting user queries to the document repository. During the indexing phase, a corpus of documents $\mathcal{D}$ is vectorized and stored. At inference, an incoming query q is transformed into a vector using the identical embedding model $E(\cdot)$. The system then executes a maximum inner product search (MIPS) to retrieve the optimal document d^* that maximizes the similarity score:

$$d^* = \arg\max_{d \in \mathcal{D}} \text{sim}(E(q), E(d)) \tag{2.3}$$

A robust embedding model effectively mitigates the lexical mismatch problem, ensuring high similarity scores for relevant statutes even when the query differs terminologically from the official legal text.

This capability is particularly critical in the domain of Vietnamese criminal law. While factual case summaries are typically articulated in descriptive, natural language, statutory provisions employ rigid and formal legal terminology. An optimal embedding model must therefore bridge this semantic gap, reliably mapping colloquial descriptions of events to their corresponding formal legal classifications within the vector space.

2.2 Large Language Models (LLMs)

Large Language Models (LLMs) are deep neural networks designed for natural language understanding and generation, typically comprising billions of parameters—such as the 175-billion parameter GPT-3 [6]. Modern LLMs are predominantly built upon the decoder-only Transformer architecture [7]. At the heart of this architecture is the self-attention mechanism, which allows the model to weigh the importance of different tokens in a sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{2.4}$$

where Q, K, and V represent the query, key, and value matrices respectively, and d_k is the scaling factor. This mechanism enables the model to process contextual relationships across a fixed-length token window.

At their core, LLMs are statistical models trained to perform next-token prediction. Given a sequence of text represented as discrete tokens $X = (x_1, x_2, \ldots, x_T)$ from a vocabulary $\mathcal{V}$, the joint probability of the sequence is modelled using the chain rule of probability:

$$P_\theta(X) = \prod_{i=1}^{T} P_\theta(x_i \mid x_1, \ldots, x_{i-1}) \tag{2.5}$$

where θ represents the model parameters. During the self-supervised pre-training phase, the model learns grammar, factual knowledge, and reasoning capabilities by minimising the negative log-likelihood (cross-entropy) loss over vast corpora of text:

$$\mathcal{L}_{\text{PT}}(\theta) = -\sum_{i=1}^{T} \log P_\theta(x_i \mid x_{<i}) \tag{2.6}$$

While pre-training creates a highly capable base model, it simply learns to continue text sequences statistically rather than follow user instructions. To bridge this gap, Supervised Fine-Tuning (SFT) is applied. During SFT, the model is trained on curated pairs of instructions X and desired responses $Y = (y_1, y_2, \ldots, y_M)$. The objective shifts to maximising the conditional probability of the response given the prompt:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{j=1}^{M} \log P_\theta(y_j \mid X, y_{<j}) \tag{2.7}$$

To further align the model's behaviour with human preferences, Reinforcement Learning from Human Feedback (RLHF) is commonly employed. A separate reward model $r_\phi(X, Y)$ is trained on human preference data to score responses. The LLM policy is then optimised—often via Proximal Policy Optimization (PPO)—to maximise the expected reward while penalising divergence from the base model.

During inference, the generation process is fundamentally autoregressive. The input prompt X is tokenised into numerical representations, and the model computes a probability distribution over the vocabulary for the next token. At timestep t, the token is sampled (or greedily selected via argmax) from the distribution:

$$\hat{x}_t \sim P_\theta(x_t \mid X, \hat{x}_1, \ldots, \hat{x}_{t-1}) \tag{2.8}$$

This predicted token $\hat{x}_t$ is then appended to the sequence, and the process repeats until a special end-of-sequence (EOS) token is generated or a maximum length is reached.

Because the output distribution P_θ is directly conditioned on the input context, prompt engineering plays a critical role in controlling model behaviour. By providing zero-shot instructions or few-shot exemplars, users can steer the statistical generation. For complex tasks, Chain-of-Thought (CoT) prompting [8] encourages the model to generate intermediate reasoning steps Z, effectively factoring the generation into $P(Y \mid X, Z)\, P(Z \mid X)$, which significantly improves performance on logic-heavy tasks.

Despite their capabilities, LLMs possess inherent limitations. Because they lack a persistent state (statelessness), context must be continuously re-injected into the prompt. Furthermore, their knowledge is bounded by their training data. When queried on domain-specific facts, the model may attempt to maximise generation probability by fabricating plausible-sounding but factually incorrect text—a phenomenon known as *hallucination*.

These limitations are particularly critical in the legal domain, where accuracy, traceability, and strict adherence to official statutes are mandatory. Therefore, relying solely on $P_\theta(Y \mid X)$ is insufficient for Vietnamese criminal judgement prediction. To mitigate hallucinations and ensure factual grounding, this project introduces a Retrieval-Augmented Generation (RAG) framework. By retrieving relevant statutes and precedents (denoted as R), the predictive distribution is conditioned on external verified knowledge, evaluating $P_\theta(Y \mid X, R)$ rather than relying purely on the model's internal parametric memory.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a framework that mitigates the limitations of parametric memory by coupling a Large Language Model (LLM) with an external information retrieval system [9]. Rather than relying solely on the weights θ learned during pre-training

to generate a response Y given a query X, RAG dynamically fetches a set of relevant documents R from an external corpus. The model's generation is then conditioned on both the query and the retrieved evidence, effectively modelling $P_\theta(Y \mid X, R)$. This paradigm is essential for tasks demanding domain-specific accuracy, verifiability, and up-to-date knowledge. Figure 2.1 illustrates the canonical Naive-RAG pipeline.

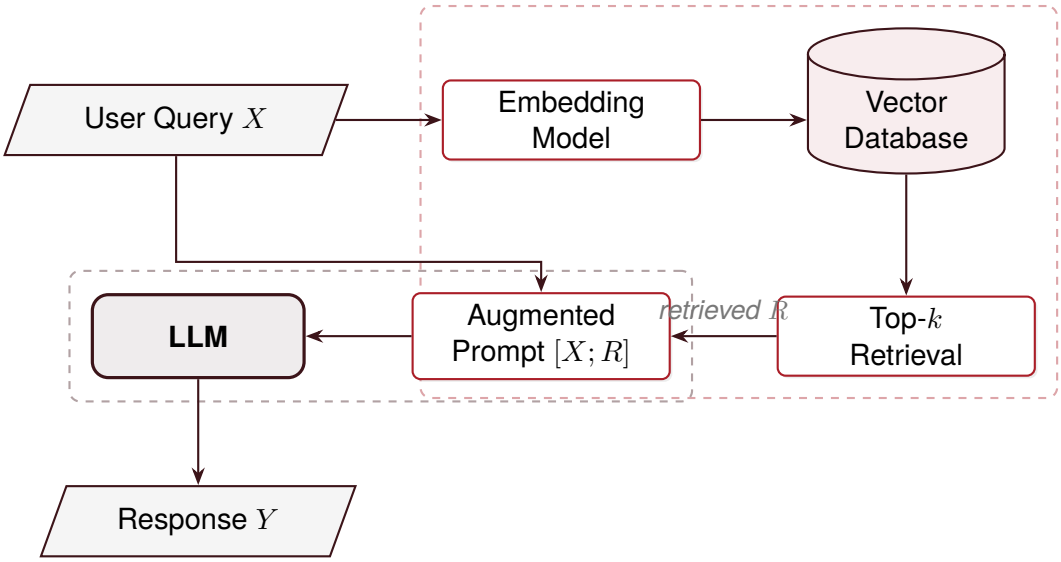

Figure 2.1. Overview of the Naive Retrieval-Augmented Generation pipeline. The retriever embeds the query, searches a vector database for the most relevant chunks R, and prepends them to the prompt before generation [9].

The standard architecture, often referred to as *Naive RAG* [10], operates in two primary phases. The first is *offline indexing*, where a corpus of documents is partitioned into smaller, manageable chunks. These chunks are transformed into dense vector representations via an embedding model and stored in a vector database. The second phase occurs during inference. The user query is embedded into the same continuous vector space, and the database retrieves the top-k most semantically similar chunks based on a distance metric (e.g., cosine similarity). These retrieved chunks R are concatenated with the original query X to form an augmented prompt, enabling the LLM to generate an empirically grounded response.

Despite its conceptual simplicity, Naive RAG is heavily bottlenecked by retrieval quality. If the retrieved context is noisy, incomplete, or irrelevant, the LLM remains susceptible to generating inaccurate outputs. Furthermore, the chunking strategy presents a critical trade-off: chunks that are too granular may strip away vital semantic context, whereas overly large chunks introduce noise and quickly exhaust the LLM's finite context window. In the legal domain, where the interpretation of a statute can pivot on a single clause or conditional phrase, these retrieval inaccuracies are particularly detrimental.

To overcome these limitations, *Advanced RAG* architectures introduce optimisations across the entire pipeline [10]. Pre-retrieval strategies enhance the input through query rewriting, expansion, or the decomposition of complex questions into multi-step sub-queries. Post-retrieval mechanisms often employ cross-encoder models to rerank candidate documents or utilise context compression to filter out extraneous text before prompt formulation. Additionally, iterative or multi-hop retrieval systems allow the LLM to execute sequential searches based on intermediate reasoning steps, which is vital for synthesising answers from multiple distinct sources.

The RAG architecture is uniquely suited for legal AI applications [11]. Legal judgement prediction inherently requires strict adherence to statutory texts and historical precedents, rendering purely parametric generation inadequate. Moreover, RAG allows the legal knowledge base to be updated dynamically as laws and judicial guidelines evolve, bypassing the computationally expensive need to retrain the LLM. In this project, a RAG framework is deployed to retrieve specific Vietnamese criminal statutes and analogous historical cases, ensuring that the model's judgements are strictly anchored in concrete, verifiable legal evidence.

2.4 Optical Character Recognition (OCR)

Optical Character Recognition (OCR) is the computational process of transcribing images of typed, handwritten, or printed text into machine-encoded strings. Formally, OCR can be modeled as a mapping function $f : \mathbb{R}^{H \times W \times C} \rightarrow \mathcal{V}^*$, where an input document image of height H, width W, and C channels is mapped to a sequence of discrete tokens from a defined vocabulary $\mathcal{V}$. In empirical datasets, documents frequently exist in unstructured visual formats, such as scanned PDFs or photographs. Bridging this modality gap is essential, as visual text cannot be directly parsed, queried, or ingested by natural language processing (NLP) architectures.

A standard OCR pipeline operates through a sequence of discrete computational phases. Let I represent the raw input image. The end-to-end extraction process can be formulated as a composite function:

$$S = f_{\text{post}}(f_{\text{recog}}(f_{\text{detect}}(f_{\text{pre}}(I)))) \tag{2.9}$$

where f_{pre} denotes preprocessing operations (e.g., noise attenuation, binarization, and affine transformations for deskewing). The detection module f_{detect} localizes text regions, f_{recog} decodes the visual features within those regions into text, and f_{post} applies heuristic rules or language models for error correction and structural reconstruction, yielding the final text sequence S.

Historically, OCR architectures bifurcated the task into independent detection and recognition modules. Text detection generates a set of bounding boxes $B = \{b_1, b_2, \ldots, b_k\}$, where each box $b_i = (x_i, y_i, w_i, h_i)$ isolates a textual instance. Subsequently, the recognition module predicts a character sequence for each cropped coordinate space. While effective for documents with homogenous typography, this bipartite approach degrades when applied to complex legal documents. Elements such as headers, marginalia, tables, official stamps, signatures, and irregular spacing introduce significant structural noise if layout semantics are ignored during extraction.

The advent of deep learning has fundamentally shifted OCR paradigms towards holistic Document Layout Analysis (DLA). Modern neural architectures leverage convolutional and transformer-based networks to maintain spatial topologies alongside textual content. Rather than merely extracting raw, unanchored strings, these systems construct a hierarchical representation of the document, identifying the topological reading order and classifying structural entities (e.g., lists, tables, headings). This spatial awareness is critical; the semantic interpretation of a legal text is often inherently tied to its geometric position and formatting, rendering purely linear text extraction insufficient.

The fidelity of OCR extraction directly constrains the efficacy of downstream NLP algorithms. Because subsequent computational layers assume clean token sequences, transcription anomalies—such as character substitutions, boundary truncation, or spurious line

breaks—introduce cascading errors. In jurisprudence, a single character perturbation within a statutory citation, a defendant's name, a penal duration, or a monetary fine can drastically alter the semantic payload of the extracted case fact. Consequently, OCR functions not merely as a trivial preprocessing step, but as a foundational data-structuring bottleneck.

Within the scope of this research, OCR is deployed to digitize Vietnamese criminal judgments sourced from the Supreme People's Court of Vietnam. These appellate and trial documents are predominantly distributed as PDF files exhibiting high variance in internal encoding—ranging from native digital text to low-fidelity scanned images. The OCR framework normalizes this heterogeneous corpus into a machine-readable format, subsequently structuring the output into serialized JSON payloads. This structured data acts as the basis for isolating salient legal entities, including defendant demographics, factual summaries, judicial reasoning, applied statutes, and sentencing parameters.

Given the rigid morphological structure and specialized taxonomy of Vietnamese legal discourse, OCR hallucination mitigation is paramount. To address this, the raw OCR output is subjected to a rigorous post-processing pipeline designed to filter extraneous page artifacts, normalize typographical inconsistencies, and reconstruct logical sentence boundaries. Ultimately, this extraction and purification process translates opaque, image-based court records into a highly structured case law database, forming the empirical substrate required for the subsequent Retrieval-Augmented Generation (RAG) and Judgment Prediction pipelines.

CHAPTER 3

Proposed Methodology

In this chapter, we present the proposed methodology and architecture for the legal judgment prediction system. Section 3.1 outlines the overall methodological approach for the problem. Section 3.2 describes the detailed process of constructing a structured dataset of Vietnamese criminal judgments from raw PDF files. Section 3.3 explains the design of the vector database and retrieval architecture. Finally, Section 3.4 discusses the reasoning design for the LLM-based prediction component, showing how the retrieved legal context is integrated into the prediction process.

3.1 Methodological Overview

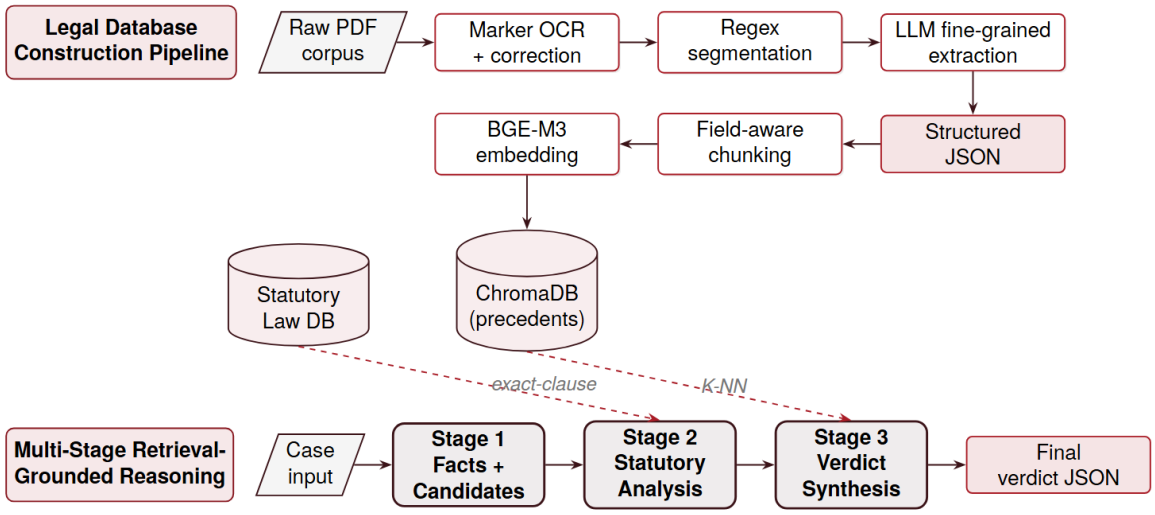

Figure 3.1. High-level methodology overview. The *Legal Database Construction Pipeline* (top) ingests raw PDFs, produces a structured JSON corpus, and indexes selected fields into ChromaDB. The *Multi-Stage Retrieval-Grounded Reasoning Pipeline* (bottom) consumes a case input through three sequential stages; Stage 2 is grounded in the statutory law database via exact-clause lookup, while Stage 3 is grounded in ChromaDB via dense K-NN retrieval.

The architecture of the Vietnamese criminal judgment prediction system is based on a Retrieval-Augmented Generation (RAG) framework integrated with a multi-stage Reason-and-Act pipeline [12]. This design brings two key advantages to the legal domain. First, it enables a clear separation of reasoning responsibilities by decomposing the complex prediction task into distinct, sequential stages: factual extraction, statutory legal analysis, precedent retrieval, and final structured verdict generation. This structured decomposition prevents the system from committing immediately to a single charge without first considering alternatives, and it allows each intermediate reasoning step to be inspected independently. Second, the design ensures

that all predictions are tightly grounded in retrieved statutory laws and historical precedents rather than relying solely on the pre-trained memory of the Large Language Model (LLM). This significantly reduces the risk of legal hallucination and ensures traceability.

The implementation of the system is divided into two primary subsystems: the *Legal Database Construction Pipeline* and the *Retrieval-Grounded Reasoning Pipeline*. Their interaction is depicted in Figure 3.1.

1. **Legal Database Construction Pipeline:** To provide a reliable foundation for RAG, the system first constructs a structured case law and statutory database from raw legal documents. This is handled by a dedicated data processing pipeline:
    - **Data Scraping and OCR:** Raw first-instance criminal judgment PDFs are automatically collected from the Supreme People's Court of Vietnam using Selenium WebDriver. Because these documents are often scanned images, the system applies the Marker Optical Character Recognition (OCR) model to extract the text into Markdown format. The text then undergoes a conservative cleanup pass using a Vietnamese text correction model to fix common OCR errors.
    - **Data Post-Processing:** The cleaned Markdown text is processed through deterministic regex segmentation to isolate major judgment boundaries, such as defendant details, case content, court reasoning, and final decisions. A fine-grained LLM extraction stage (gemma-4-31b-it provided by Google AI Studio) is then used to convert these complex narrative paragraphs into highly structured JSON records containing discrete fields for applied legal clauses, mitigating/aggravating circumstances, and specific penalties.
    - **Vector Database Initialization:** To facilitate semantic search, specific structured fields—such as the court's core legal reasoning and defendant-specific sentencing factors—are chunked and converted into dense numerical vectors using the BAAI/bge-m3 embedding model. These embeddings are stored in a persistent ChromaDB collection, allowing the system to retrieve analogous historical cases based on semantic similarity rather than just keyword matching.
2. **The Multi-Stage Retrieval-Grounded Reasoning Pipeline:** At the core of the prediction process is a central controller that orchestrates the Reasoning-and-Act loop. Instead of issuing a single, direct generation prompt, the controller routes the case through three predefined and logically constrained stages:
    - **Stage 1: Input Preparation and Candidate Extraction:** The LLM receives the structured defendant information and a synthetic case summary. It is tasked with extracting discrete facts (e.g., relevant conduct, property value, harm, prior convictions, admissions) and proposing two to five plausible candidate offenses under the Vietnamese Criminal Code.
    - **Stage 2: Statutory-Law Retrieval and Legal Analysis:** Based on the candidate offenses generated in Stage 1, the system's exact-clause retriever fetches the corresponding legal texts from the statutory law database, alongside a mandatory set of supporting criminal-law articles (such as those governing recidivism or accomplice liability). The LLM performs a rigorous legal analysis over the extracted facts and retrieved laws to explicitly select the most likely offense, identify the applicable sentencing bracket, and formally reject or downgrade unsuitable candidates.

- **Stage 3: Precedent Retrieval and Verdict Synproject:** Once the controlling statutory article is selected, the system triggers the vector database. It retrieves similar past cases—strictly filtered to match the selected statutory article—for analogy, as well as specific historical instances of mitigating or aggravating factors for sentencing calibration. Finally, the LLM processes all accumulated evidence—the original case facts, the legal analysis, the retrieved statutes, and the precedent summaries—to produce the final structured verdict. This includes the precise predicted charge, applied legal clauses, imprisonment penalty, monetary fines, and civil liabilities.

Through this hierarchical, step-by-step pipeline, the system mimics a structured legal reasoning process, ensuring that the final judgment predictions are logically justified, legally accurate, and calibrated against historical court decisions.

3.2 Dataset Construction Methodology

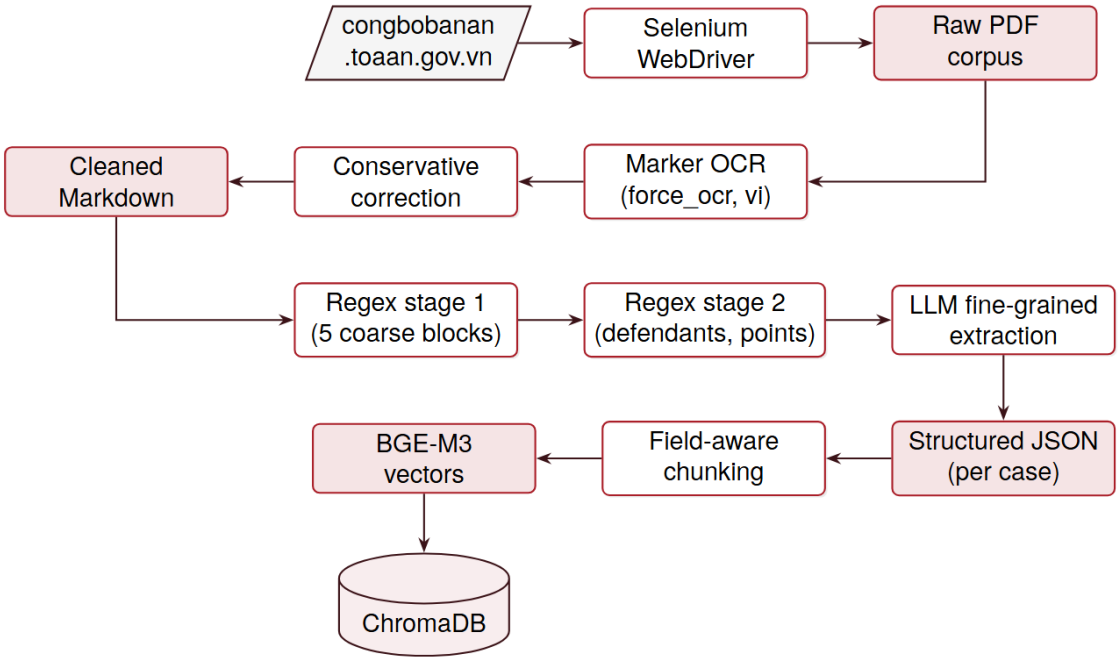

Figure 3.2. Dataset construction methodology, organised into three sequential methodological stages. Stage A collects raw PDFs from the public court portal under three categorical filters. Stage B converts scanned PDFs into clean Vietnamese Markdown via Marker OCR followed by a strict word-level mBART correction. Stage C transforms the cleaned Markdown into a fine-grained JSON record through a coarse regex split, an intermediate regex split, and a deterministic LLM extraction constrained by a Pydantic response schema.

Constructing a structured dataset of Vietnamese criminal judgments is a primary contribution of this project. Because Vietnamese legal judgments are predominantly published as unstructured, image-based PDF files, a robust pipeline is required to transition this raw data into a machine-readable format suitable for retrieval-augmented generation. This methodology is divided into data collection, automated optical character recognition (OCR), and multi-stage data structuring.

3.2.1 Data Collection

The data collection workflow employs browser automation to compile public criminal caselaw documents from the portal of the Supreme People's Court of Vietnam: `https://congbobanan.toaan.gov.vn/0tat1cvn/ban-an-quyet-dinh`. A semi-automated crawling strategy using Selenium WebDriver is utilized to navigate the dynamic web interface, managing pagination, search filters, and embedded PDF downloads.

To establish a high-quality jurisprudence corpus, the extraction is constrained by three categorical filters:

1. **"Cấp Tòa án" (Court Level):** Configured to "TAND cấp tỉnh" (Provincial People's Court). This restricts the retrieval to judgments issued at the provincial tier, securing a corpus of significant regional jurisprudence while defining a practical boundary for batch processing the thousands of resulting documents.
2. **"Cấp giải quyết/xét xử" (Trial Level):** Configured to "Sơ thẩm" (First Instance). Targeting first-instance trials ensures the collected dataset captures the fundamental legal reasoning, the initial establishment of facts, and the primary sentencing logic before any potential appellate review.
3. **"Loại vụ/việc" (Case Type):** Configured to "Hình sự" (Criminal) (Criminal). This is the definitive filtering criterion, strictly confining the extracted documents to criminal proceedings and aligning the scraped data with the core domain of the knowledge base.

Each downloaded PDF is assigned a normalized, deterministic identifier derived from metadata patterns matching the trial date, province, and stable URL segments, ensuring an organized foundation for downstream processing.

3.2.2 Automated OCR Pipeline

Due to the scanned nature of the collected PDFs, an automated OCR pipeline using the Marker text recognition framework is applied to convert visual documents into Markdown text. The pipeline is configured to enforce full-page OCR, preserve pagination, and utilize Vietnamese language hints to maximize recognition accuracy on GPU-accelerated infrastructure.

To address inherent OCR artifacts—such as missing diacritics, merged words, or spelling errors—a conservative post-OCR correction mechanism is integrated. The pipeline applies a fine-tuned mBART-based text correction model (bmd1905/vietnamese-correction-v2 [13]) to systematically rectify syntax errors. This step is strictly constrained by rule-based filters to merge corrections at the word level, aggressively rejecting structural rewrites, insertions, or deletions, thereby maintaining the strict legal integrity of the original judgment.

3.2.3 Data Structuring and Schema Design

While caselaw documents exhibit a standardized macro-level template, the development of a high-fidelity dataset with precise labels remains a complex undertaking due to inconsistent phrasing and non-standardized drafting styles across different texts. A primary obstacle in natural language processing for this domain is the accurate extraction of structured data from dense, unstructured paragraphs. This difficulty is heavily compounded by the lack of uniform typographical markers denoting section boundaries, as well as the reliance on deeply nested

legal citations (e.g., “theo quy định tại các điểm a, b khoản 341”). Such intricacies require robust parsing strategies to reliably map these complex textual relationships into a predefined, highly structured data schema. Transforming unstructured markdown into a highly organized JSON schema is critical in constructing a dataset for accurate queries and precise reasoning. The methodology utilizes a three-stage post-processing pipeline:

The first step use deterministic pattern matching isolates major logical boundaries within the text, splitting the document into foundational blocks. Because Vietnamese court judgments are semi-structured, regular expressions provide an efficient rule-based approach to recognize repeated structural patterns like section titles and participant labels. To ensure this matching is tolerant of Optical Character Recognition (OCR) diacritic errors, the text first undergoes a normalization step that strips Markdown heading markers, removes emphasis characters, decomposes Unicode, and removes Vietnamese diacritics. This initial stage successfully extracts five primary blocks: Defendant Information (Bi_cao), capturing the defendants and their personal details; Related Parties (Lien_quan), an optional block for participants like victims, witnesses, and civil plaintiffs/defendants; Case Content (NOI_DUNG_VU_AN), detailing facts, investigation results, and indictments; Court Reasoning (NHAN_DINH); and the Final Decision (QUYET_DINH), which houses the applied legal citations, offenses, penalties, court fees, and civil liabilities.

Second step detect further regular expressions subdivide the coarse blocks into smaller, deterministic fields. Within the defendant blocks, the system isolates individual defendants by splitting multi-defendant blocks at numerical prefixes and bolded names. The broad case content section (NOI_DUNG_VU_AN) is divided into two distinct sub-fields: the investigation narrative (Qua_trinh_dieu_tra) and the party conclusions/prosecutor recommendations (Ket_luan_cac_ben), which are separated by identifying specific indictment or prosecutor markers in the text. Additionally, the court’s reasoning section (NHAN_DINH) is systematically split into indexed points to isolate specific legal arguments; if the system encounters reasoning points with duplicate indices, it appends their text together rather than discarding the duplicate data.

The final step produce the complete JSON record by feeding complex paragraphs that resist deterministic parsing through a Large Language Model (gemma-4-31b-it) via an API service layer. In this final stage, the LLM is invoked with a temperature of zero alongside a structured Pydantic response schema to guarantee deterministic, JSON-formatted outputs. The LLM performs highly nuanced extractions, such as parsing out fine-grained defendant background details (e.g., prior convictions, education, and temporary detention dates), distributing hierarchical legal citations into explicit point–clause–article formats, and classifying exact imprisonment lengths, fines, and civil liabilities. Crucially, the model also identifies specific mitigating and aggravating circumstances and links them directly to the integer index of the court-reasoning point where they were discussed.

A key architectural design in this schema is the explicit separation of complex attributes, particularly individual defendant traits. The LLM is prompted to parse merged text into granular JSON fields such as full name, date of birth, prior convictions (Tien_An), criminal history (Tien_Su), and specific mitigating/aggravating circumstances. By isolating these attributes per defendant rather than preserving them as monolithic paragraphs, the system drastically mitigates the risk of overlapping facts in multi-defendant trials and significantly improves the precision of subsequent similarity matching and penalty calibration.

Ultimately, this rigorous, multi-stage methodology transforms a raw corpus of over $\langle$*insert quantity*$\rangle$ unstructured, image-based public caselaw documents into a meticulously structured,

machine-readable database. By bridging the gap between noisy, unstandardized provincial court PDFs and a high-fidelity JSON schema, this pipeline establishes a foundational dataset that addresses the historical scarcity of large-scale, domain-specific training data in the Vietnamese legal sector.

The resulting dataset is explicitly optimized for advanced deep learning and natural language processing tasks. The strict architectural separation of granular fields—most notably the disambiguation of individual defendant background traits (nhân thân) and the isolation of hierarchical legal citations—provides highly precise, noise-free targets for embedding models and semantic similarity matching. This structural clarity directly mitigates the hallucination risks and context-overlap issues typically encountered when feeding monolithic text chunks into LLMs. Furthermore, the standardized quality and scale of the data provide a reliable, structured benchmark necessary for evaluating, fine-tuning, and optimizing future Vietnamese-specific language models on complex downstream tasks, such as legal judgment prediction and algorithmic penalty calibration.

3.3 Retrieval Architecture

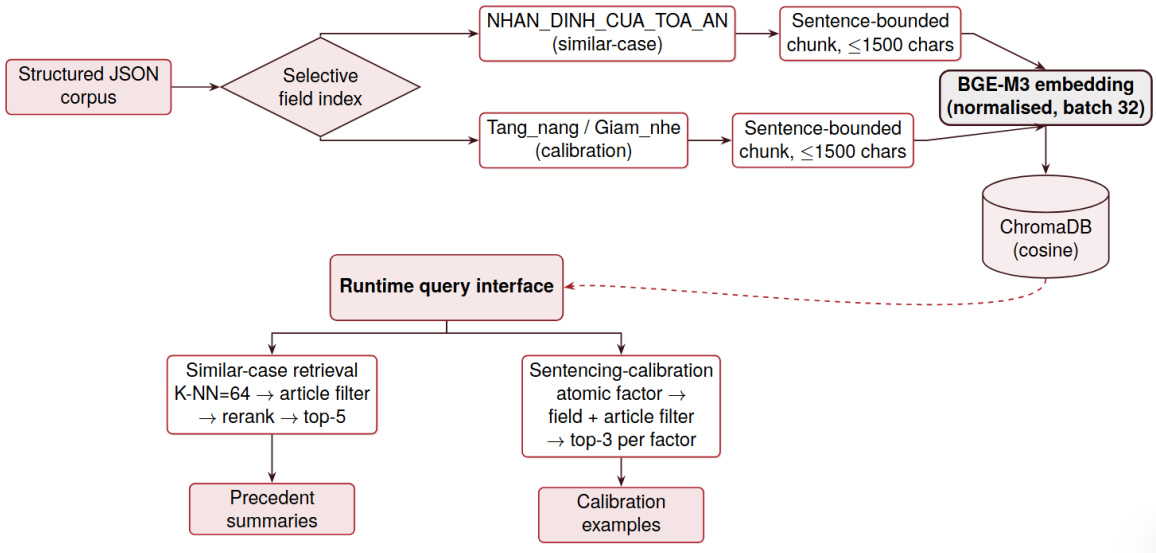

Figure 3.3. Retrieval architecture. The structured JSON corpus is selectively indexed at field granularity: court reasoning text (`NHAN_DINH_CUA_TOA_AN`) supports similar-case retrieval, while mitigating and aggravating fields (`Tang_nang`, `Giam_nhe`) support sentencing calibration. Chunks are bounded by sentence splits and a 1500-character cap, embedded with BGE-M3, and persisted in ChromaDB under cosine similarity. At query time, two specialised retrievers expose precedent summaries and calibration examples back to the reasoning pipeline.

The retrieval architecture constitutes the core infrastructure of the proposed RAG system, designed to ground the Large Language Model's reasoning engine in highly relevant statutory texts and historical jurisprudence. By leveraging dense vector embeddings for semantic search, the system transcends rigid keyword matching to ensure high retrieval accuracy based on contextual meaning. This semantic abstraction is paramount in the legal domain, as it successfully overcomes the inherent lexical variance across provincial court judgments, ensuring

that identical legal concepts, mitigating circumstances, and factual elements are reliably retrieved regardless of inconsistent drafting styles.

To optimize retrieval, the text is not embedded at the document level. Instead, a targeted chunking strategy is applied to specific structured fields. For example, the court's core legal reasoning (typically NHAN_DINH_CUA_TOA_AN) is indexed to facilitate similar-case retrieval, whereas distinct mitigating and aggravating factors (Tang_nang, Giam_nhe) are indexed to support precise sentencing calibration. Text splitting is enforced at natural sentence boundaries with a maximum threshold of 1,500 characters. These text chunks are transformed into dense numerical vectors using the multilingual BAAI/bge-m3 embedding model [14] via the Sentence-Transformers framework. The normalized vectors are subsequently stored in a ChromaDB persistent local database, allowing for rapid K-Nearest Neighbors (K-NN) searches utilizing cosine distance as the primary similarity metric.

3.4 LLM Reasoning Design

To prevent hallucination and enforce logical legal derivation, the reasoning process abandons the one-shot prompting approach in favor of a Multi-Stage Reason-and-Act pipeline. The retrieved context is strictly compartmentalized across three sequential reasoning stages:

1. **Stage 1: Fact Extraction and Candidate Generation:** The process begins by having the language model analyze the initial case information, specifically the defendant's background and a summary of the events. Instead of immediately deciding on a final charge, the model breaks down the scenario to extract key factual elements, such as the defendant's conduct, the resulting harm, prior convictions, and any potential mitigating or aggravating circumstances. Using these extracted facts, the system brainstorms a shortlist of plausible offenses. This step acts as an exploratory phase, ensuring the system considers multiple legal possibilities and avoids jumping to premature conclusions.

2. **Stage 2: Statutory-Law Retrieval and Legal Analysis:** Once the potential charges are identified, the system consults its legal database to retrieve the exact text of those specific laws, alongside fundamental criminal provisions that govern general rules like sentencing bases or recidivism. The language model then conducts a formal legal analysis by carefully evaluating the extracted facts against the definitions in the retrieved statutes. It eliminates the incorrect charges and confidently selects the single most applicable offense and its corresponding sentencing framework. This stage ensures that the core decision is strictly grounded in written law rather than the model's internal memory.

3. **Stage 3: Precedent Retrieval and Verdict Synproject:** After the exact law and sentencing bracket are established, the system searches its database of past court decisions to find historical cases that match the current scenario and fall under the newly selected legal article. It looks for past cases that share similar factual circumstances for analogy, as well as historical examples of how previous courts weighed identical mitigating or aggravating factors. Finally, the model synthesizes all of this gathered context—the original facts, the statutory analysis, and the historical precedents—to produce a well-reasoned, comprehensive final verdict that includes the specific charge and the appropriate penalty.

This strict orchestration ensures that statutory law universally governs the output, while vector-retrieved precedents are utilized exclusively for qualitative analogy and sentence length calibration, which is crucial to accurately approximate the exact penalty length.

CHAPTER 4

Design and Implementation

4.1 Database construction

The database construction pipeline begins with the collection of Vietnamese criminal judgments in PDF format. Each document undergoes optical character recognition before being segmented into structured JSON fields through a combination of deterministic rules and large language model (LLM) processing.

4.1.1 Data scraping

The judgment documents are collected from the public portal of the Supreme People's Court of Vietnam: `https://congbobanan.toaan.gov.vn/0tat1cvn/ban-an-quyet-dinh`

Three categorical filters are applied to narrow the scope of collection:

1. **Cấp Toàn án (Court Level):** Set to "TAND cấp tỉnh" (Provincial People's Court). Restricting collection to provincial-tier judgments yields a corpus of regionally significant decisions while keeping the total document count tractable for batch processing.
2. **Cấp giải quyết/xét xử (Trial Level):** Set to "Sơ thẩm" (First Instance). First-instance judgments are targeted because they contain the primary narrative of fact-finding, the initial application of law, and the original sentence rationale prior to any appellate review.
3. **Loại vụ/việc (Case Type):** Set to "Hình sự" (Criminal). This filter confines the collected corpus strictly to criminal proceedings, matching the intended domain of the knowledge base.

Because the portal does not expose a structured API, the collection process relies on a semi-automated browser workflow:

1. Launch a browser session with automatic PDF downloading enabled.
2. Navigate to the judgment publication page.
3. Manually apply the criminal-case filter, specify a date range, and submit the search.
4. Iterate through result pages, recording each judgment title and its detail URL.
5. Deduplicate the collected URLs and persist the title–URL pairs.
6. For each detail page, locate the attached PDF, trigger the download, wait for completion, and assign a normalized filename.
7. Cross-check downloaded files against the collected URL list and retry any gaps. Pages lacking a standard PDF attachment or exceeding the timeout threshold are skipped.

Each downloaded PDF is assigned a normalized identifier derived from its title and detail URL:

<date>-<province>-<url-id>.pdf

The date component is extracted from phrases matching "ngày DD/MM/YYYY". The province component is captured from text following location markers such as "tỉnh", "thành phố", "TP", or "tại", with diacritics removed and spaces replaced by underscores. The URL id is taken from the first non-empty path segment of the detail URL. The regular expression patterns used during this process are:

- `ngày\s+(\d{1,2}[/\- ]\d{1,2}[/\- ]\d{4})`: Date contained in the judgment title.
- `(?:tỉnh|thành phố|TP\.?|tại)\s+([A-ZĐ][\w\s]+)`: Province or city phrase contained in the title.

4.1.2 OCR

Text extraction from the collected PDFs is performed using the Marker framework, which produces Markdown output for each document. The following options were configured for the OCR run:

- `force_ocr`: True; OCR is applied to every page, including those that appear digitally typeset.
- `paginate_output`: True; page boundary markers are retained in the output.
- `languages`: vi; Vietnamese is specified as the recognition language hint.
- `batch_multiplier`: 2; internal GPU batch size is scaled up for throughput.
- `max_pages`: None; all pages of each document are processed.
- `device`: `cuda:0` when a CUDA-capable GPU is available; CPU otherwise.

Processing was configured for a Kaggle T4 GPU with 16 GB VRAM. For each input PDF, the framework produces a Markdown file; non-empty outputs are saved along with a status record and elapsed processing time. GPU memory is released between documents to prevent out-of-memory errors — a necessary precaution given that Marker loads multiple Surya submodels simultaneously, and residual activations from one document can accumulate across a large batch. Previously processed files are skipped on reruns to support incremental processing, since the full corpus spans thousands of documents and a single interrupted run should not require restarting from scratch.

The raw Markdown output then undergoes a conservative correction pass using the local Hugging Face model `bmd1905/vietnamese-correction-v2`. Proposed corrections are accepted only when they correspond to diacritic restoration or minor OCR spelling fixes at the word level. This conservative threshold is deliberate: legal documents contain proper nouns, defendant names, case numbers, and monetary values that a general-purpose language model may misidentify as errors and silently alter, potentially corrupting the exact fields that downstream extraction depends on. Insertions, deletions, punctuation changes, and substantive rewrites are therefore rejected unconditionally. Page-break markers are stripped from the

corrected output as a final cleanup step. The active regular expressions during this correction pass are:

- `^\s*$|^\{?\d+\}?-{3,}\s*$`: Skip blank lines and page-break lines during correction.
- `^(#{1,6}\s+|(?:\s*[-*+]\s+)+|\s*\d+\.\s+|\s*>\s+)?`: Separate a Markdown prefix from heading content.
- `(?<=[.;!?])\s+` and `(?<=,)\s+`: Split long headings into bounded segments.
- `\*{1,3}`: Remove Markdown emphasis before model inference.
- `\n*\s*\{?\d+\}?-{3,}\s*\n*`: Remove page-break markers from corrected output.

4.1.3 Data post-processing

The corrected Markdown for each judgment passes through a regex-based segmentation step followed by LLM processing to produce a structured JSON representation. Although judgment layout varies across courts and time periods, the extraction workflow targets the following logical content blocks:

- the trial date and case introduction;
- one or more defendants and their personal details;
- optional participants such as victims, related persons, witnesses, civil plaintiffs, and civil defendants;
- a case-content section describing facts, investigation results, indictments, and prosecutor recommendations;
- the court's reasoning section;
- the court's decision section, including legal citations, offenses, penalties, court fees, civil liability, supplementary penalties, and physical-evidence handling when present.

Post-processing proceeds through three stages:

1. Coarse section extraction locates the major structural boundaries of each judgment and writes one JSON object per successfully parsed document.
2. Intermediate extraction subdivides the coarse blocks into smaller, deterministically derived fields.
3. Fine-grained LLM extraction converts complex prose sections into structured defendant and verdict fields.

Prior to pattern matching, both regex stages normalise the text by stripping Markdown heading markers, removing emphasis and backslash characters, decomposing Unicode, removing Vietnamese diacritics, converting D-with-stroke to plain D, and uppercasing where appropriate. This normalisation makes section boundary detection tolerant of diacritic OCR errors.

a) First regex stage

The first regex stage extracts five blocks:

- **Bi_cao:** Extracted text section that contain one or more defendants and their personal details.
- **Optional Lien_quan:** Extracted text section that contain optional participants such as victims, related persons, witnesses, civil plaintiffs, and civil defendants.
- **NOI_DUNG_VU_AN:** Extracted text section that contain case-content section describing facts, investigation results, indictments, and prosecutor recommendations.
- **NHAN_DINH:** Extracted text section that contain the court's reasoning section.
- **QUYET_DINH:** Extracted text section that contain the court's final decision section, including legal citations, offenses, penalties, court fees, civil liability, supplementary penalties, and physical-evidence handling when present.

The regex code that is used to extract text sections is listed below:

- `^#{1,6}\s*`: Remove a Markdown heading prefix before matching.
- `^\s*#{1,6}\s`: Recognize heading-style section lines.
- `NGAY\s+\d{1,2}\s+THANG\s+\d{1,2}\s+NAM\s+\d{4}`: Trial-session line written with day, month, and year words.
- `NGAY\s+\d{1,2}/\d{1,2}/\d{4}`: Trial-session line written with a numeric date.
- `(?:TRONG\s+(?:CAC\s+)?|TU\s+)NGAY`: Trial-session prefix variants such as "during the day(s)" or "from day".
- `^(?:-\s*)?(?:BI|NGUOI\s+BI)\s+HAI\b`: Start of victim information when no related-person heading appears first.
- `^(?:-\s*)?NGUYEN\s+DON\s+DAN\s+SU\b`: Start of civil-plaintiff information.
- `^(?:-\s*)?BI\s+DON\s+DAN\s+SU\b`: Start of civil-defendant information.
- `^(?:-\s*)?NGUOI\s+LAM\s+CHUNG\b`: Start of witness information.
- `QUYEN LOI` and `LIEN QUAN`: Keyword test for the related-person section.
- `NOI DUNG` and `VU AN`: Keyword test for the case-content heading.
- `NHAN DINH` and `TOA AN`: fallback `NHAN DINH` and `HOI DONG`; final fallback standalone `NHAN DINH`: Keyword tests for the court-reasoning heading.
- `NHAN DINH NHU SAU`: Last-resort inline reasoning marker.
- `QUYET DINH` with heading-style validation: Decision heading. Longer body phrases that merely contain these words are rejected.
- `^\s*[\[\(]?\s*\d+\s*[\]\)]?\s*\.?\s*` and `^VE\s+`: Remove numbering and a leading `VE` before validating the decision heading

The JSON result from this stage has the following format:

```
{
  "filename": "Original Markdown filename, including the
               .md extension.",
  "Bi_cao": "Raw Markdown/text section from the trial-date
             line up to the related-parties section if
             present, otherwise up to NOI DUNG VU AN.",
  "Lien_quan": "Raw Markdown/text section for other
                participants. Null when no related-party
                boundary is detected.",
  "NOI_DUNG_VU_AN": "Raw Markdown/text section from the
                     NOI DUNG VU AN heading up to the
                     NHAN DINH section.",
  "NHAN_DINH": "Raw Markdown/text section from the
                NHAN DINH CUA TOA AN heading up to the
                QUYET DINH heading.",
  "QUYET_DINH": "Raw Markdown/text section from the
                 QUYET DINH heading to the end of the
                 Markdown file.",
  "_missing": "List of missing mandatory extraction fields.",
  "_lines": {
    "trial_date": "Zero-based source line index.",
    "lien_quan": "Zero-based source line index, or -1.",
    "noi_dung": "Zero-based source line index, or -1.",
    "nhan_dinh": "Zero-based source line index, or -1.",
    "quyet_dinh": "Zero-based source line index, or -1."
  }
}
```

b) Second regex stage

The second regex stage extracts defendant blocks, splits the case-content section into investigation content and party conclusions, and splits court reasoning into indexed points. If reasoning points have duplicate indices, their text is appended instead of discarded. The regex code that is used to extract text sections is listed below:

- `^[-\s]*\d+\.\s+`: Possible numbered defendant start.
- `sinh\s+(?:ngày|năm)`: Birth-information confirmation for a defendant start.
- `^[-\s]*\*\*[^\*]+\*\*`: Possible bold defendant name.
- `(?:đối với|bị cáo) [:\s]`: Possible direct defendant mention.
- `^[-\s]*(?:\*\*)?\d+\s*[.\)]\s*`: Split a multi-defendant block at numbered starts.
- `\n\s*\(\s*[Cc]ác\s+bị\s+cáo\s+.*?\)\s*$`: Remove a trailing collective-attendance line from a defendant block.
- `^(?:#{1,6}\s*)?(?:\*{1,2})?N[ỘÔO]I\s+DUNG\s+V[ỤU]\s+[AÁ]N\s*:?\s*(?:\*{1,2})?\s*\n?`: Remove the case-content heading before further splitting.

- `(?:tại\s+)?(?:bản\s+)?cáo\s+trạng(?:\s+số)?\s*[:\.]?\s*\d`: Start of an indictment statement.
- `(?:tại\s+)?(?:bản\s+)?cáo\s+trạng(?:\s+của)?\s+viện\s+kiểm\s+sát`: Start of an indictment statement naming the prosecutor's office.
- `đại\s+di[ệễ]n\s+viện\s+kiểm\s+sát.*?(?:phát\s+biểu|thực\s+hành|có\s+ý\s+kiến|giữ\s+nguyên|rút|tham\s+gia)`: Start of a prosecutor statement.
- `tại\s+phiên\s+tòa(?:.*?đại\s+di[ệễ]n)?\s+viện\s+kiểm\s+sát`: Start of a prosecutor statement at the trial.
- `(?:tai\s+)?(?:ban\s+)?cao\s+trang(?:\s+so)?\s*[:\. ]*\d`: Normalized fallback for a numbered indictment statement.
- `(?:tai\s+)?(?:ban\s+)?cao\s+trang(?:\s+cua)?\s+vien\s+kiem\s+sat`: Normalized fallback for an indictment statement naming the prosecutor's office.
- `dai\s+dien\s+vien\s+kiem\s+sat.*?(?:phat\s+bieu|thuc\s+hanh|co\s+y\s+kien|giu\s+nguyen|rut|tham\s+gia)`: Normalized fallback for a prosecutor statement.
- `tai\s+phien\s+toa(?:.*?dai\s+dien)?\s+vien\s+kiem\s+sat`: Normalized fallback for a prosecutor statement at the trial.
- `(?m)^\s*(?:#{1,6}\s*)?(?:[-*+]\s*)?(?:\*\*\*)?\s*\[\s*(\d+)\s*\](?:\*\*\*)?\s*(?:[.\]\-:]\s*)?`: Split the court's reasoning into indexed points such as [1] and [2].

The indictment and prosecutor-marker patterns are first applied to the original text and then to a lowercased, diacritic-stripped version. The JSON output of the second stage has the following format:

```
{
  "filename": "Original Markdown filename from Stage 1.",
  "case_type": "Detected case type: so_tham, phuc_tham,
               or unknown.",
  "Danh_sach_bi_cao": [
    "One raw text block per defendant extracted from the
     Stage 1 Bi_cao section."
  ],
  "Nguoi_tham_gia": "Raw related-participants text. Null
                     when Stage 1 Lien_quan is null.",
  "Noi_dung_vu_an": {
    "Qua_trinh_dieu_tra": "Investigation/factual narrative
                           before the indictment marker.",
    "Ket_luan_cac_ben": "Prosecution conclusions, party
                         statements, and trial-position
                         text."
  },
  "Nhan_dinh_cua_toa_an": {
    "[1]": "Text of court-assessment point [1].",
    "[2]": "Text of court-assessment point [2].",
    "[n]": "Additional dynamic numbered assessment points."
  },
```

```
  "QUYET_DINH": "Raw decision/verdict text from Stage 1 QUYET_DINH. In
    the current pipeline this is not split into sub-fields and is not
    passed through the clean_quyet_dinh helper."
}
```

c) Fine-grained LLM extraction

The final stage extracts fine-grained case law detail that is not possible using deterministic pattern matching. This includes constructing a detail schema for defendant information, extract law articles applied by the court, extract final verdict for each defendant and identify charges, fine, imprisonment length. The script submits relevant fields that contain the required data to the LLM model using API request and builds a JSON output from the LLM response. The list of field sent to the LLM from the second stage is listed below:

- **Danh_sach_bi_cao**: defendant blocks;
- **Noi_dung_vu_an.Ket_luan_cac_ben**: party conclusions and prosecutor recommendation text;
- **Nhan_dinh_cua_toa_an**: indexed court-reasoning points;
- **QUYET_DINH**: the decision text.

The primary extraction model is `gemma-4-31b-it`. It is called with temperature zero and a structured JSON response schema. Returned JSON is validated against the schema before it is merged into the final record. The script concatenates the output with a prompt that instructs the model to:

- extract defendant personal details;
- extract one prosecutor-recommendation object per defendant;
- create one court-verdict object per defendant;
- distribute hierarchical legal citations into explicit point–clause–article objects;
- classify offenses, imprisonment, monetary fines, court fees, civil liability, and supplementary punishments;
- identify aggravating and mitigating circumstances;
- attach the court-reasoning point index to supplementary punishment, aggravating circumstance, and mitigating circumstance fields.

The final output is a structured JSON file with the schema shown below (abbreviated for readability):

```
{
  "THONG_TIN_CHUNG": {
    "Ma_Ban_An": "Case identifier derived from filename.",
    "Thong_Tin_Bi_Cao": [
      {
        "Ho_Ten": "Defendant full name.",
        "Ngay_Sinh": "Date of birth. Omitted when null.",
        "Noi_Cu_Tru": "Residence/address. Omitted when null.",
```

```
        "Nghe_Nghiep": "Occupation. Omitted when null.",
        "Trinh_Do_Van_Hoa": "Education level. Omitted when null.",
        "Dan_Toc": "Ethnicity. Omitted when null.",
        "Gioi_Tinh": "Gender. Omitted when null.",
        "Ton_Giao": "Religion. Omitted when null.",
        "Quoc_Tich": "Nationality. Omitted when null.",
        "Hoan_Canh_Gia_Dinh": "Family circumstance summary.",
        "Tien_An": "Previous-conviction summary.",
        "Tien_Su": "Prior criminal/admin history.",
        "Chi_Tiet_Nhan_Than": "Additional personal details.",
        "Ngay_Tam_Giam": "Temporary detention date.",
        "Trang_Thai_Co_Mat": "Presence status at trial."
      }
    ],
    "Thong_Tin_Nguoi_Tham_Gia_To_Tung": "Raw participant text."
  },
  "NOI_DUNG_VU_AN": "Raw factual/investigation narrative.",
  "NHAN_DINH_CUA_TOA_AN": {
    "[1]": "Court-assessment point [1].",
    "[2]": "Court-assessment point [2].",
    "[n]": "Additional numbered assessment points."
  },
  "De_Nghi_Cua_Vien_Kiem_Sat": [
    {
      "Bi_Cao": "Defendant name.",
      "Pham_Toi": ["Offense name(s)."],
      "Phat_Tu": "Imprisonment recommendation text."
    }
  ],
  "PHAN_QUYET_CUA_TOA_SO_THAM": [
    {
      "Bi_Cao": "Defendant name.",
      "Can_Cu_Dieu_Luat": [
        {
          "Diem": "Legal point (e.g., a or s).",
          "Khoan": "Legal clause number.",
          "Dieu": "Legal article number.",
          "Bo_Luat_Va_Van_Ban_Khac": "Law identifier."
        }
      ],
      "Pham_Toi": ["Offense name(s)."],
      "Phat_Tu": "Imprisonment sentence.",
      "Phat_Tien": "Monetary fine.",
      "An_Phi": "Court-fee decision text.",
      "Hinh_Phat_Bo_Sung": "Supplementary punishment.",
      "Hinh_Phat_Bo_Sung_index": "NHAN_DINH point index.",
      "Trach_Nhiem_Dan_Su": "Civil liability decision.",
      "Tang_nang": "Aggravating-circumstance text.",
      "Tang_nang_index": "NHAN_DINH point index.",
      "Giam_nhe": "Mitigating-circumstance text.",
      "Giam_nhe_index": "NHAN_DINH point index."
    }
  ],
  "totals": {
    "prompt_tokens": "Sum of prompt tokens.",
    "completion_tokens": "Sum of completion tokens.",
    "total_tokens": "Sum of total tokens.",
    "cost_usd": "Total estimated USD cost."
```

```
    }
}
```

4.1.4 Data filtering

Structured case records are filtered to remove cases with missing fields, missmatch between defendant information and structured verdict, missing critical information such as final verdict, court reasoning, court conclusion. The final dataset contains 4855 cases, of which 361 cases is use for evaluation and 4494 cases is used as the database for retrieval

4.2 Vector Database

The vector database forms the retrieval backbone of the RAG architecture, but its design reflects a deliberate choice not to index whole documents. A full-document embedding would blend the court's legal reasoning with procedural boilerplate, defendant biographical data, and decision text, diluting the semantic signal that makes retrieval useful. Instead the indexing pipeline targets only the fields that directly serve each retrieval objective, keeping the embedded content semantically focused and the retrieved context interpretable.

4.2.1 Data Indexing and Chunking Strategy

Rather than embedding the entire document of a legal case, the data pipeline selectively indexes specific fields tailored for distinct retrieval objectives. The court's core legal reasoning and factual assessments, typically located within the `NHAN_DINH_CUA_TOA_AN` field, are indexed to support similar-case retrieval. Conversely, defendant-specific aggravating and mitigating factors, extracted from the `PHAN_QUYET_CUA_TOA_SO_THAM.Tang_nang` and `Giam_nhe` fields, are indexed to assist in sentencing calibration.

The chunking process iterates through the JSON files on a field-by-field basis. For nested structures, such as defendant-specific verdicts, each list item is processed independently and annotated with structural metadata, including the record index and the corresponding defendant's name. To maintain optimal embedding quality, text splitting enforces a maximum chunk length of 1,500 characters. Fields exceeding this threshold are initially split along natural sentence boundaries (e.g., periods, exclamation marks, or question marks). In cases where a single sentence still surpasses the character limit, a hard-slice by character count is applied. Each resulting chunk is assigned a deterministic identifier derived from the document ID and the field name. This deterministic approach ensures resumable indexing and mitigates the risk of duplicate entries during the database compilation phase.

4.2.2 Embedding Generation and Storage

To transform the textual chunks into dense numerical vectors, the system employs the BAAI/bge-m3 model [14]. This model is deployed using the Sentence-Transformers framework [5] and accelerated via a CUDA-enabled GPU. During the generation phase, embeddings are processed in batches of 32 with normalization enabled.

The storage layer uses ChromaDB configured as a persistent local database, with cosine distance as the similarity metric. Since all embeddings are normalised, cosine distance is directly convertible to a similarity score at query time. Each stored record contains the deterministic chunk ID, the embedding vector, the source text, and an associated metadata dictionary.

4.2.3 Retrieval Mechanisms

During the evaluation workflow, the runtime query interface executes two specialized retrieval mechanisms:

- **Similar-Case Retrieval:** A composite query is constructed from the current case's factual narrative and the defendant's profile. An initial K-NN search retrieves 64 candidate cases from the database. A hard filter then removes candidates whose ground-truth labels do not include the statutory article identified in the preceding reasoning stage. The remaining candidates are deduplicated by document ID and reranked using a weighted combination of vector similarity, lexical token overlap, and the presence of a custodial sentence for the target offense. The top five cases are formatted as compact summaries and injected into the generation prompt.
- **Sentencing-Calibration Retrieval:** This mechanism retrieves historical examples of how courts have evaluated specific mitigating or aggravating factors. Each atomic factor statement is issued as an independent query, filtered to match both the relevant metadata field (`PHAN_QUYET_CUA_TOA_SO_THAM.Giam_nhe` or `PHAN_QUYET_CUA_TOA_SO_THAM.Tang_nang`) and the target statutory article. Retrieved chunks are mapped back to the corresponding defendant-verdict record, and up to three examples per factor are returned.

Notably, the raw vector results are not fed directly into the LLM. Instead, they are parsed into high-level, structured representations that concisely summarize the factual profile, court reasoning, and final sentence. This abstraction ensures that the retrieved context remains bounded, auditable, and separated from the low-level semantic search operations.

4.3 LLM API service

The reasoning subsystem uses a unified LLM service layer so that the prompting and validation logic remain independent from any single vendor SDK. The service accepts a provider identifier, a model name, a system prompt, a user prompt, and a Pydantic output schema, then returns a validated structured object together with token-usage metadata.

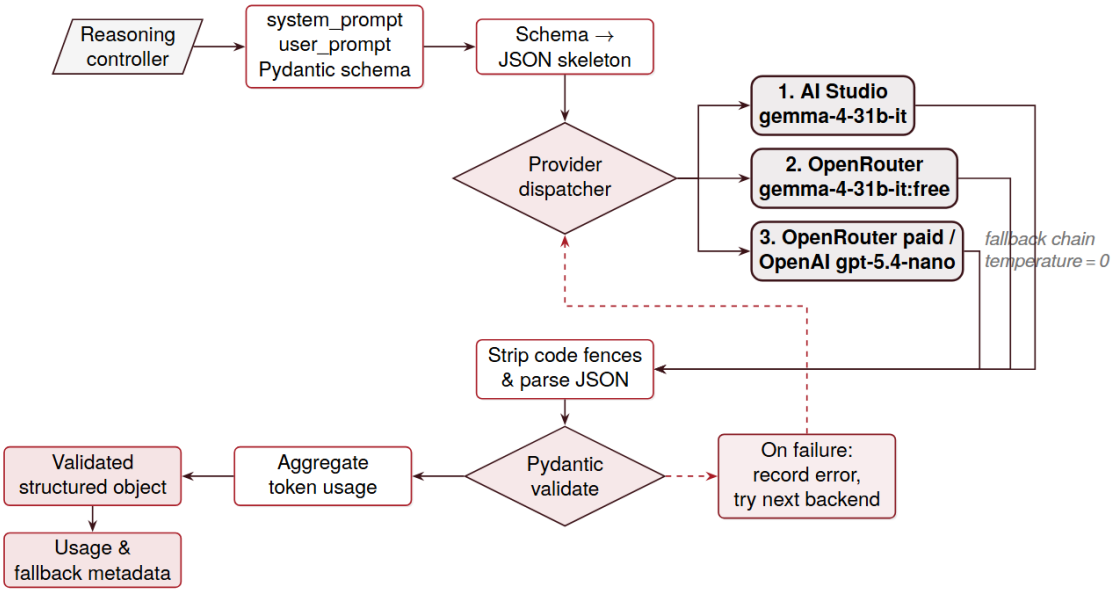

Figure 4.1. Unified LLM API service.

The reasoning controller submits a prompt bundle plus a Pydantic schema; the schema is rendered as a JSON skeleton and the provider dispatcher selects one of three backends (AI Studio, OpenRouter, or OpenAI), all invoked at temperature zero. Responses are stripped of Markdown fences, parsed, and validated against the schema before the call returns a structured object together with token-usage and fallback metadata. Validation or transport failures fall through a deterministic backoff chain that is recorded in the usage record.

4.3.1 Provider abstraction and default models

The provider interface supports three backends:

- `aistudio`: Google AI Studio with default model `gemma-4-31b-it`;
- `openrouter`: OpenRouter with default model `google/gemma-4-31b-it:free`;
- `openai`: OpenAI with default model `gpt-5.4-nano`.

All backends are called with temperature zero to produce deterministic, schema-compatible output. OpenRouter and OpenAI are accessed through the OpenAI-compatible chat-completions interface; Google AI Studio uses the `google.genai` client. Provider credentials are read from environment variables (`GOOGLE_API_KEY`, `OPENROUTER_API_KEY`, and `OPENAI_API_KEY`). The AI Studio request timeout is configurable via `LLM_REQUEST_TIMEOUT` and defaults to 90 seconds. The AI Studio client is instantiated lazily and cached for reuse across calls.

This abstraction means that the reasoning pipeline can call a single helper function without knowing whether the request will be served by AI Studio, OpenRouter, or OpenAI. The provider-specific implementations are therefore limited to authentication, request submission, and extraction of the returned token-usage information, while prompt construction and post-validation remain shared.

The normalized usage record returned by the wrapper contains a stable set of fields regardless of provider. An example from a saved reasoning run is shown below:

```
{
  "prompt_tokens": 26523,
  "completion_tokens": 3262,
  "total_tokens": 32703,
  "provider": "aistudio",
  "model": "gemma-4-31b-it",
  "raw_response_preview": "{\n \"legal_analysis\": ...",
  "fallback_attempts": [
    {"provider": "openrouter",
     "model": "google/gemma-4-31b-it:free"},
    {"provider": "aistudio",
     "model": "gemma-4-31b-it"},
    {"provider": "openrouter",
     "model": "google/gemma-4-31b-it:paid"}
  ],
  "fallback_errors": []
}
```


This metadata is important during evaluation because it allows later analysis of which backend produced the final answer, how expensive the call was in token terms, and whether any fallback path was needed before obtaining a valid structured response.

4.3.2 Structured output validation

The API service is designed for structured generation rather than free-form text completion. Each prompt is paired with a Pydantic schema such as `ReasonActAnalysisOutput`, `ReasonActLegalAnalysis`, or `ReasonActFinalOutput`. A helper routine converts the schema into a prompt-friendly JSON skeleton so that the prompt specification and the runtime parser remain synchronized.

After a response is received, the service strips any surrounding Markdown code fences, parses the remaining content as JSON, and validates it against the target schema. A JSON null response is converted to an empty object before validation, allowing optional-field schemas to be interpreted as "no value extracted" rather than a hard parsing error. Each successful call records prompt tokens, completion tokens, total tokens, and a short preview of the raw response.

For readability, the prompt builder does not expose raw Python type annotations to the model. Instead, it converts each Pydantic model into a prompt-oriented JSON skeleton with field descriptions. A simplified view of the `ReasonActAnalysisOutput` contract is shown below:

```
{
  "facts": {
    "defendants": ["string"],
    "conduct": "string|null",
    "dates": ["string"],
    "victims": ["string"],
    "property_value": "string|null",
    "harm": "string|null",
    "admissions": "string|null",
    "prior_convictions": "string|null",
    "mitigation_signals": ["string"],
```

```
    "aggravation_signals": ["string"],
    "stated_facts": ["string"],
    "inferred_facts": ["string"],
    "missing_facts": ["string"]
  },
  "candidates": [
    {
      "Dieu": "string|null",
      "Khoan": "string|null",
      "Diem": "string|null",
      "offence_name": "string|null",
      "search_query": "string|null",
      "supporting_facts": "string|null",
      "rejection_or_downgrade_reason": "string|null"
    }
  ],
  "mitigation_factors": ["string"],
  "aggravation_factors": ["string"]
}
```

The later stages use larger schemas, especially `ReasonActLegalAnalysis` and `ReasonActFinalOutput`, but they follow the same principle: every required output field is declared before the request is sent, and every returned field is validated before it is merged into the reasoning trace.

4.3.3 Fallback and fault tolerance

Long evaluation runs may involve hundreds of structured API calls, so the implementation includes a provider-fallback mechanism. When fallback is enabled, the default retry order is:

1. OpenRouter free-tier model;
2. AI Studio default model;
3. OpenRouter paid-tier model.

When the preferred backend is AI Studio or OpenAI, the retry order is adjusted to attempt that provider first. Duplicate provider–model pairs within the sequence are skipped automatically. Each failed attempt records its error message, and the final usage record includes both the successful provider information and the full list of failed attempts, making evaluation logs auditable and simplifying failure diagnosis.

This is particularly valuable because the failure modes are heterogeneous. A request may fail before model execution due to a missing API key, during transport due to a timeout, or after generation because the model returned malformed JSON or omitted required fields. By treating all of these as ordinary exceptions inside the wrapper, the outer reasoning pipeline can keep a simple control flow: either it receives a validated object, or it receives a single aggregated failure after all configured fallbacks are exhausted.

4.4 LLM reasoning process

The main judgment-generation workflow is implemented in `rag/generation/reasoning_act.py`. Instead of issuing a single prompt, the system decomposes reasoning into multiple structured stages. This design separates factual extraction, statutory-law selection, precedent retrieval, and final verdict synproject, allowing each step to be inspected independently.

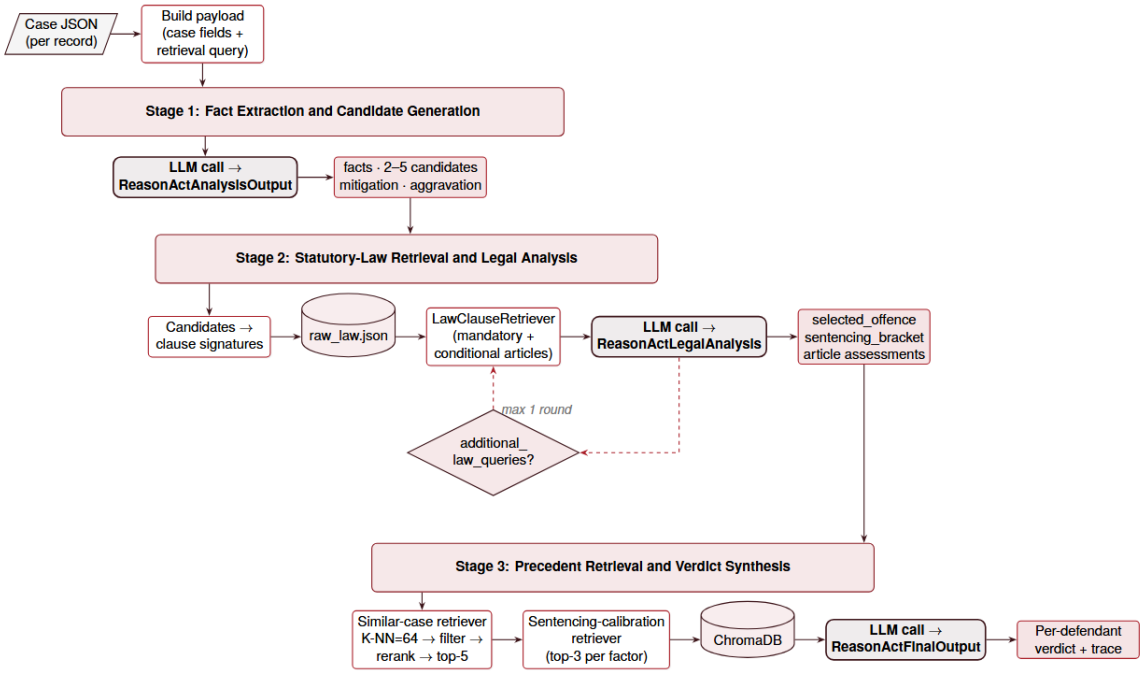

Figure 4.2. Reason-and-Act reasoning process.

Each case JSON record is routed through three sequential reasoning stages, each backed by a structured LLM call. Stage 1 extracts facts and proposes 2–5 candidate offences. Stage 2 resolves each candidate to a precise clause via the deterministic `LawClauseRetriever` over `raw_law.json` and may issue at most one additional-law round before producing a legal analysis. Stage 3 queries ChromaDB for analogous precedents and per-factor sentencing calibration examples, after which the LLM synthesises the final structured verdict together with the full reasoning trace.

4.4.1 Stage 1: input preparation and candidate extraction

The controller first builds a case payload from a configurable subset of the case JSON fields. By default, the reasoning function uses the defendant information block and the synthetic case summary, while the evaluation script may override these fields from the command line. A second configurable field list is concatenated to form the vector-retrieval query.

The first prompt is not just a free-text instruction. It is a structured JSON payload containing the document identifier, the selected case fields, format notes for multi-defendant summaries, the extraction task list, and the output schema. A condensed example is shown below:

```
{
  "doc_id": "12-2024-HSST-example",
  "case_fields": {
    "THONG_TIN_CHUNG.Thong_Tin_Bi_Cao":
      "[defendant background text]",
    "Synthetic_summary":
      "Bi cao ... dung thu doan gian doi ..."
  },
  "input_format": {
    "Synthetic_summary":
      "May be a JSON array encoded as a string.",
    "per_defendant_requirement":
      "Do not merge one defendant's facts into another."
  },
  "task": [
    "Extract structured facts.",
    "List mitigating factors as atomic statements.",
    "List aggravating factors as atomic statements.",
    "Generate 2 to 5 plausible BLHS offence candidates."
  ],
  "output_schema": { "...ReasonActAnalysisOutput..." }
}
```

The first LLM call receives this payload and returns a `ReasonActAnalysisOutput` object. This object contains: extracted facts and missing facts; two to five candidate BLHS offenses; atomic mitigating-factor statements; and atomic aggravating-factor statements.

When the synthetic summary contains one first-person story per defendant, the prompt explicitly instructs the model to keep each defendant's facts separate, preserve the original names, and avoid transferring mitigating or aggravating details from one defendant to another.

A real saved output from this stage, abridged for space, has the following shape:

```
{
  "facts": {
    "defendants": ["Thào Thị C", "Sồng A T", "Sùng A C1"],
    "conduct": "Các bị cáo cùng phối hợp mua bán trái phép
        chất ma túy...",
    "dates": ["10/10/2024", "11/10/2024", "12/10/2024"],
    "property_value": "Giá mua: 50 triệu; Giá bán: 90 triệu;
        Giá dự kiến: 160 triệu",
    "harm": "Phát tán 350,40 gam Heroin ra thị trường.",
    "admissions": "Cả ba bị cáo đều thành khẩn khai báo và
        hối hận.",
    "prior_convictions": "Sồng A T từng bị kết án năm 2015
        nhưng đã được xóa án tích.",
    "missing_facts": [
      "Thông tin về việc các bị cáo có bị cưỡng ép hay
        không."
    ]
  },
  "candidates": [
```

```
    {
      "Dieu": "250",
      "Khoan": "4",
      "Diem": null,
      "offence_name": "Tội mua bán trái phép chất ma túy"
    }
  ],
  "mitigation_factors": [
      "Thào Thị C là người dân tộc thiểu số.",
      "Cả ba bị cáo đều thành khẩn khai báo."
    ],
  "aggravation_factors": [
      "Khối lượng ma túy giao dịch lớn (350,40 gam Heroin)."
    ]
    }
```


4.4.2 Stage 2: statutory-law retrieval and legal analysis

Each candidate offense is converted into a clause signature and resolved by the `LawCla useRetriever`. The retriever normalizes flexible references such as `174-4-a`, `51-2`, or textual forms containing Dieu, Khoan, and Diem, then returns the exact legal text from `raw_law.json`. If only a full article is requested, the returned text includes the article together with its nested clauses and points; if a specific clause is requested, the output is narrowed to that clause and its points.

The retriever itself returns a structured dictionary rather than a plain text string. A short exact-clause lookup example taken directly from the codebase is shown below:


```
{
  "query": "51-1-s",
  "normalized": "51-1-s",
  "found": true,
  "level": "diem",
  "text": "[dieu 51] Các tình tiết giảm nhẹ trách nhiệm hình
      sự
[khoan 1] Các tình tiết sau đây là tình tiết giảm nhẹ trách
      nhiệm hình sự:
[diem s] Người phạm tội thành khẩn khai báo, ăn năn hối
      cải;",
  "dieu": "51",
  "khoan": "1",
  "diem": "s"
}
```


The system also retrieves supporting BLHS articles that are repeatedly needed for sentencing analysis. The mandatory set is: 38, 50, 51, 52, 53, 54, 55, 56, 57, 58, 65, and 47. Additional supporting articles are included when keyword rules suggest issues such as compensation, property handling, commercial-entity liability, or juvenile defendants.

The second LLM call receives the extracted facts, candidate offenses, retrieved offense articles, and supporting articles, and returns a `ReasonActLegalAnalysis` object. The prompt forces the model to select an offense and a sentencing bracket from retrieved statutory law, reject or downgrade the remaining candidates, classify each supporting article as `applicable`, `fact_dependent`, `not_applicable`, or `not_retrieved`, and identify missing facts that could still alter the outcome.

A representative legal-analysis response from a saved run is shown below in abridged form:


```
{
  "selected_offence": {
    "Dieu": "174",
    "Khoan": "4",
    "Diem": "a",
    "offence_name": "Toi lua dao chiem doat tai san"
  },
  "rejected_candidates": [
    {
      "Dieu": "360",
      "offence_name": "Toi thieu trach nhiem gay hau
                       qua nghiem trong",
      "rejection_or_downgrade_reason":
        "Primary criminal act is fraudulent appropriation."
    }
  ],
  "supporting_article_assessments": [
    {
      "article": "51",
      "status": "applicable",
      "factual_trigger": "Confession, repentance, and
                          compensation are stated.",
      "explanation": "Mitigating factors under khoan 1
                      Dieu 51 are supported."
    },
    {
      "article": "53",
      "status": "not_applicable",
      "factual_trigger": "Defendants have no prior
                          convictions.",
      "explanation": "Recidivism is not established."
    }
  ],
  "sentencing_bracket": "Phat tu tu 12 nam den 20 nam
                         hoac tu chung than",
  "missing_facts": [
    "Specific dates of the fraudulent transactions."
  ],
  "confidence": "High."
}
```


If the retrieved law appears incomplete, the model may emit a list of exact `additional_law_queries`. The controller then deterministically fetches only the new signatures and reruns the legal-analysis stage. This loop is bounded by `max_additional_law_rounds`, which defaults to one extra round.

Missing mandatory supporting-article assessments are backfilled by rule-based checks so

that the final reasoning trace always covers the mandatory article set.

4.4.3 Stage 3: precedent retrieval and verdict synproject

The vector database is queried for similar past cases only after the controlling statutory article has been confirmed in Stage 2. This sequencing is deliberate: retrieved precedents may inform sentencing calibration, but the selection of the applicable law article must be grounded in statutory text rather than inferred from historical decisions.

The similar-case retriever first performs a broad vector search of up to 64 candidates. It then removes non-case rows, filters candidates whose stored ground-truth labels do not contain the selected BLHS article, deduplicates by document ID, and reranks the remaining cases using a combination of vector similarity, lexical token overlap, and the presence of a prison sentence for the selected article. Up to five compact summaries are kept.

The sentencing-calibration retriever operates at a finer granularity. Each atomic mitigating or aggravating statement is used as a separate query, and only chunks from the verdict fields `PHAN_QUYET_CUA_TOA_SO_THAM.Giam_nhe` or `PHAN_QUYET_CUA_TOA_SO_THAM.Tang_nang` are accepted. Retrieved examples are mapped back to the original defendant-level verdict record so the final prompt can compare the current case against historical prosecution proposals, court-recognized factors, and actual imprisonment terms.

The final LLM prompt therefore receives five different categories of evidence:

1. the original case payload;
2. the selected and rejected statutory-law analysis;
3. the retrieved offense and supporting articles;
4. similar-case summaries for analogy;
5. sentencing-calibration cases for quantitative comparison.

The final LLM call receives the case payload, the validated legal analysis, the retrieved offense and supporting articles, the similar-case summaries, and the sentencing-calibration examples. The prompt explicitly states that statutory law controls the result and that retrieved cases may be used only for analogy and sentencing calibration. The model returns a `ReasonActFinalOutput` containing both the legal-analysis trace and a structured `GenerationOutput` with per-defendant offense, applied clauses, imprisonment term, fine, civil liability, and physical-evidence handling.

An abridged real prediction from the saved evaluation output is listed below. The original record contained three defendants; only one defendant entry is shown here to keep the report readable:


```
{
  "prediction": {
    "defendants": [
      {
        "Bi_Cao": "Thao Thi C",
        "Phan_Tich_Phap_Ly": "Bi cao dong vai tro chu
          chot ... co to chuc ... thanh khan khai bao ...",
        "Toi_Danh": "Toi mua ban trai phep chat ma tuy",
        "Applied_Law_Clauses": [
          {
```

```
            "Dieu": "250", "Khoan": "4", "Diem": "b",
            "Tinh_tiet_ap_dung": "Mua ban trai phep
              Heroine voi khoi luong 350,40 gam.",
            "Bo_Luat_Va_Van_Ban_Khac": "BLHS"
          },
          {
            "Dieu": "51", "Khoan": "1", "Diem": "s",
            "Tinh_tiet_ap_dung": "Thanh khan khai bao,
              an nan hoi cai.",
            "Bo_Luat_Va_Van_Ban_Khac": "BLHS"
          },
          {
            "Dieu": "52", "Khoan": "1", "Diem": "a",
            "Tinh_tiet_ap_dung": "Pham toi co to chuc.",
            "Bo_Luat_Va_Van_Ban_Khac": "BLHS"
          }
        ],
        "Phat_Tu": "20 nam tu",
        "Phat_Tien": null,
        "Trach_Nhiem_Dan_Su": null
      }
    ],
    "Xu_Ly_Vat_Chung": "Tich thu tieu huy toan bo 350,40
      gam Heroin thu giu duoc."
  }
}
```


4.4.4 Trace recording and failure handling

In addition to the final structured prediction, the controller persists the complete reasoning trace at every stage: extracted facts, candidate offenses, requested additional laws, retrieved articles, similar cases, sentencing-calibration cases, supporting-article assessments, missing facts, and per-call token-usage records. This trace is returned to the evaluation pipeline together with the structured prediction so that legal errors can be diagnosed stage by stage. A safe wrapper catches JSON parsing and schema-validation failures and records them as `parse_error`; other runtime exceptions are recorded as `generation_error`.

The saved trace is itself a structured research artifact. A small excerpt from one real trace is shown below:


```
{
  "selected_offence": {
    "Dieu": "250", "Khoan": "4", "Diem": "b"
  },
  "similar_cases": [
    {
      "doc_id": "04-07-2025-Son_La-2ta1870336t1cvn",
      "matched_offence_article": "250",
      "sentence": "tu chung than"
    }
  ],
  "sentencing_calibration_cases": [
    {
      "factor_type": "mitigation",
      "query_factor": "Thao Thi C la nguoi dan toc
```

```
                        thieu so.",
      "doc_id": "24-06-2025-Son_La-2ta1859818t1cvn",
      "defendant_name": "Lau Thi A",
      "court_sentence": "tu chung than"
    }
  ]
}
```


This trace makes the final verdict auditable in a way that a one-shot generation cannot. A reviewer can inspect which offense article was selected, which analogous cases were considered, how sentencing factors were calibrated, and whether the model failed because of missing facts, incorrect law retrieval, or a schema-formatting error. For a legal-generation system, this intermediate visibility is not merely convenient; it is a core part of the implementation design.

CHAPTER 5

Analytic problems and solutions

This chapter presents the evaluation setup and experimental results for the proposed legal judgement prediction system. Three approaches are compared:

1. **Retrieval-Grounded Reasoning:** The proposed multi-stage pipeline. It extracts structured facts, retrieves exact statutory law text, evaluates supporting sentencing provisions, retrieves similar past cases for analogy and sentencing calibration, and generates a final structured verdict prediction.
2. **Single-Step Reasoning:** The selected case fields are submitted directly to the language model in a single structured generation call, without retrieving law text or similar past cases. This baseline measures the raw legal reasoning capability of the language model alone.
3. **Past-Case Reasoning:** Retrieves similar cases from the training set, mines their legal clauses, fetches explicit article-level law text, and makes one structured generation call with the retrieved context. This combines RAG with LLM generation but omits the multi-step candidate evaluation and sentencing analysis of the proposed method.

All three systems are evaluated on the same output schema and scored against the ground-truth verdict fields extracted from the structured case records.

5.1 Evaluation methodology and metrics

5.1.1 Evaluation scope

The main experimental pipeline is implemented in `rag/evaluation/reasoning_act_eval.py`. This script evaluates the multi-stage reasoning workflow described in the previous chapter rather than a single one-shot prompt. For each test case, the evaluator:

1. loads one structured case JSON file from the test directory;
2. runs the reasoning pipeline to generate a structured verdict prediction;
3. extracts the ground-truth verdict fields from the same JSON file;
4. aligns predicted and ground-truth defendants by normalized name;
5. computes per-defendant and per-document metrics;
6. stores the prediction, reasoning trace, and intermediate diagnostics in an output JSON report.

The evaluation script saves intermediate results after each processed file and can resume a partially completed run. This incremental persistence is useful because the generation work-

flow includes multiple LLM calls, retrieval steps, and structured parsing, all of which can make a complete run long-running.

5.1.2 Evaluation input configuration

a) Model input fields

The evaluation is designed around compressed case representations rather than the full original judgment text. In the current evaluator configuration, the generation input is built from:

- `THONG_TIN_CHUNG.Thong_Tin_Bi_Cao`: structured defendant information extracted during database construction;
- `Synthetic_summary_2`: a generated synthetic summary that restates the case in concise natural language.

This design reflects the intended experimental question: whether the reasoning system can predict the final structured verdict from a compact case representation composed of defendant metadata and a generated narrative summary, rather than from the entire raw judgment.

The prompt design also supports multi-defendant cases. When the synthetic summary is encoded as a list of separate first-person narratives, the prompts explicitly instruct the model to preserve defendant separation and not transfer one defendant's facts, mitigating circumstances, or aggravating circumstances to another defendant.

b) Retrieval query fields

The LLM-generation input and the retrieval query are configured separately. In the current `reasoning_act_eval.py` configuration, the query text used for similar-case retrieval is built from:

- `Synthetic_summary`;
- `THONG_TIN_CHUNG.Thong_Tin_Bi_Cao`.

The reasoning pipeline therefore distinguishes between: (i) the fields supplied directly to the LLM for prediction; and (ii) the fields concatenated to build the semantic retrieval query. This separation allows the experiment to control the generation context and the retrieval context independently.

c) Train-time retrieval index

Before evaluation, the script prepares or loads a vector database over the training cases. In the current configuration, the indexed training fields are:

- `NHAN_DINH_CUA_TOA_AN.[2]` for similar-case retrieval;
- `PHAN_QUYET_CUA_TOA_SO_THAM.Tang_nang` for aggravation calibration;
- `PHAN_QUYET_CUA_TOA_SO_THAM.Giam_nhe` for mitigation calibration.

The default embedding model is BAAI/bge-m3 [14]. The runtime evaluation configuration keeps the retrieval model and ChromaDB collection loaded in memory so that repeated

test queries do not repeatedly reload heavy components.

d) Reasoning-time retrieval settings

The reasoning evaluator uses several configurable retrieval and control parameters:

- `broad_top_k_case = 64`: the initial candidate pool for similar-case retrieval;
- `top_k_case = 5`: the maximum number of final similar-case summaries kept after filtering and reranking;
- `top_k_per_factor = 3`: the maximum number of retrieved sentencing-calibration cases per mitigating or aggravating factor;
- `max_additional_law_rounds = 1`: the maximum number of extra legal-analysis loops after the model requests new statutory signatures.

These are evaluation-time control settings, not outcome measures. They define how much retrieval context the reasoning pipeline may inspect before producing a final structured judgment prediction.

5.1.3 Ground truth construction

a) Verdict source field

The ground truth is extracted from the structured verdict field `PHAN_QUYET_CUA_TOA_SO_THAM` contained in each test JSON file. Each item in this list is treated as one defendant-level ground-truth verdict record. For each defendant, the evaluator extracts:

- defendant name `Bi_Cao`;
- offense name `Pham_Toi`;
- imprisonment text `Phat_Tu`;
- fine text `Phat_Tien`;
- civil-liability text `Trach_Nhiem_Dan_Su`;
- legal basis list `Can_Cu_Dieu_Luat`.

The prediction side is extracted from the model's `GenerationOutput.defendants` array using the same defendant-level structure.

b) BLHS-only filtering

By default, the evaluator runs with `only_blhs = True`. This means that the legal-basis comparison ignores non-BLHS sources and scores only clauses whose `Bo_Luat_Va_Van_Ban_Khac` field is recognized as belonging to the Penal Code. This choice makes the legal-basis metric narrower and more stable, because it focuses on the criminal-law article selection task rather than mixing Penal Code citations with procedural or other supporting legal references.

c) Signature normalization

Both ground-truth and predicted legal bases are normalized into comparable signatures constructed from Dieu, Khoan, and Diem. For example:

- `174` denotes article 174;
- `174-4` denotes clause 4 of article 174;
- `174-4-a` denotes point a, clause 4, article 174.

However, the current metric implementation applies an additional reduction step before computing precision, recall, and F1. Each signature is collapsed to its Dieu component only. Therefore, the reported legal-basis metrics are currently article-level scores, even though the intermediate extraction preserves more detailed clause and point structure.

d) Defendant alignment

Predicted and ground-truth defendants are aligned by a normalized name key. The normalization procedure: trims repeated whitespace; strips Vietnamese accents; converts text to lowercase; and removes non-alphanumeric characters.

The evaluator then forms the union of normalized defendant keys appearing in either the prediction or the ground truth. This choice has two consequences: (i) missing predicted defendants remain visible and are penalized rather than silently ignored; and (ii) hallucinated extra defendants also remain visible and are penalized.

In addition to quality metrics, the evaluator stores alignment diagnostics: `matched_count`, `gt_only_count`, and `pred_only_count`.

5.1.4 Metrics

a) Legal-basis precision, recall, and F1

For each aligned defendant, the evaluator compares the predicted article set P with the ground-truth article set G. Let:

- $TP = |P \cap G|$;
- $FP = |P - G|$;
- $FN = |G - P|$.

The per-defendant legal-basis metrics are then:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.1}$$

with the conventional implementation rule that a division-by-zero case yields 0.0.

These per-defendant values are averaged within each document to produce: `law_clause_precision_macro`, `law_clause_recall_macro`, and `law_clause_f1_macro`.

After all processed documents are scored, the evaluator computes a second macro average across documents. The saved run summary reports these final metrics under the names:

`law_clause_set_precision_macro`, `law_clause_set_recall_macro`, and `law_clause_set_f1_macro`.

Although the field names refer to "clause sets," the current implementation should be interpreted as an article-level macro score because of the Dieu-only reduction step described above.

b) Imprisonment-term error

The evaluator also scores the predicted custodial sentence text `Phat_Tu`. To make free-text sentence expressions comparable, the helper `extract_imprisonment_months` converts them into an integer number of months. The conversion rules implemented in `rag/core/sentencing.py` include:

- suspended sentence, fine-only sentence, warning, and other non-custodial outcomes map to 0 months;
- life imprisonment maps to 360 months;
- a sentence range such as "tù 02 năm đn 03 năm tù" maps to the midpoint;
- if a combined sentence after *tong hop* is present, that final combined sentence is preferred;
- mixed year-month expressions are parsed before single-unit fallback rules.

For each aligned defendant, let $\hat{m}$ be the predicted number of months and m be the ground-truth number of months. The evaluator records the squared error:

$$(\hat{m} - m)^2 \tag{5.2}$$

for that defendant.

Within each document, the script computes:

$$\text{RMSE}_{\text{doc}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{m}_i - m_i)^2} \tag{5.3}$$

where n is the number of aligned defendant keys in the union of prediction and ground truth for that document.

The saved per-document metric is `phat_tu_rmse_months`. Afterward, the run summary macro-averages these document-level values and stores the result as `phat_tu_rmse_months_mac`

c) Operational counters and failure accounting

Besides the quality metrics, the evaluator also records run-level status counters: `n_processed`, `n_failed`, and `n_skipped`.

These are bookkeeping indicators rather than predictive-quality metrics, but they are important for interpreting an unfinished run. A document can fail for two main reasons:

- `parse_error`: the returned JSON could not be parsed or validated against the expected schema;

- `generation_error`: another runtime exception occurred during reasoning, retrieval, or model invocation.

Because the current full test run is not yet complete, these counters are reported here only as part of the protocol definition and not as finalized experimental outcomes.

5.1.5 Saved evaluation artifacts

The evaluation output is saved as a JSON report containing three top-level components:

- `config`: the run configuration, including input fields, query fields, retrieval settings, model/provider settings, and BLHS-only scoring mode;
- `per_doc`: one saved record per evaluated case;
- `summary`: the final aggregate metrics, written only after the evaluator finishes the full run.

Each per-document record stores: the raw structured prediction; the reasoning trace; the retrieved similar-case and sentencing-calibration context; the requested and retrieved law articles; defendant-level metric details; document-level macro metrics; and token-usage metadata for the underlying LLM calls.

This output format supports two levels of analysis. First, it supports final quantitative reporting through the summary metrics. Second, it supports error analysis by making every intermediate artifact inspectable: the input payload, selected offense, supporting-article assessments, retrieved analogous cases, and the final structured verdict.

5.1.6 Reporting policy for the unfinished run

Because the full test process is still ongoing, this chapter deliberately stops at methodology and metric definition. Final numerical tables, comparative model results, and any discussion of train/test split size should be added only after the evaluation run has completed and the saved summary section is stable.

CHAPTER 6

Conclusion and Future Work

6.1 Summary

This project introduced a Retrieval-Augmented Generation (RAG) framework integrated with a multi-stage Reason-and-Act pipeline for Vietnamese criminal judgment prediction. The end-to-end system spans the full life cycle of legal-AI processing: it scrapes raw PDF judgments from the Supreme People's Court of Vietnam, applies a Vietnamese-aware OCR and correction stage, segments and extracts the documents into a structured JSON case-law database, and indexes selected fields in a ChromaDB vector store using BGE-M3 embeddings. On top of this infrastructure, a controller orchestrates three logically constrained reasoning stages—fact extraction and candidate generation, statutory retrieval and legal analysis, and precedent retrieval and verdict synthesis—each producing a schema-validated structured artefact. The result is an auditable pipeline in which every decision can be traced back to the statutory clauses and historical cases that supported it.

6.2 Key Contributions

1. **End-to-End Data Pipeline.** A reproducible pipeline that converts raw Vietnamese criminal judgment PDFs into structured JSON records, capturing defendant profiles, case facts, judicial reasoning, statutory citations, and final penalties.
2. **Dual-Retrieval Legal Database.** A specialised retrieval substrate that supports both exact-clause lookups over the Penal Code and semantic search over historical judgments, enabling joint factual and statutory grounding.
3. **Multi-Stage Reason-and-Act Framework.** A bounded reasoning workflow that separates fact extraction, statutory selection, and precedent calibration, with mandatory supporting-article assessments and a deterministic fallback mechanism across LLM providers.
4. **Auditable Evaluation Protocol.** A reproducible evaluation harness with article-level Precision/Recall/F1 on legal citations and RMSE on imprisonment months, together with full per-stage reasoning traces for error analysis.

6.3 Limitations

Several limitations of the current system should be acknowledged. First, the system is restricted to first-instance criminal proceedings and to the Penal Code; civil, administrative, and commercial law are out of scope. Second, OCR quality is the dominant upstream con-

straint: although Marker combined with the Vietnamese correction model performs well on clean PDFs, low-quality scans remain a source of segmentation errors. Third, the article-level reduction of legal-basis metrics underestimates the granularity of the system's internal reasoning, which already operates at the point/clause level. Finally, the system is a research prototype and is not intended for unsupervised judicial decision-making.

6.4 Future Work

Future extensions of this project include:

- **Hierarchical metric reporting.** Reporting Precision/Recall/F1 at the article, clause, and point levels jointly, to better reflect the model's actual reasoning granularity.
- **Domain adaptation.** Fine-tuning the embedding model on Vietnamese legal text to improve precision in the sentencing-calibration retriever.
- **Cross-court generalisation.** Extending the dataset to appellate decisions and other jurisdictions to evaluate robustness under stylistic and procedural variation.
- **Human-in-the-loop interaction.** Adding a user-facing interface that exposes the reasoning trace so that legal experts can audit, override, or annotate intermediate decisions.
- **Comparative study.** Benchmarking the proposed multi-stage pipeline against one-shot baselines and against Advanced-RAG variants such as cross-encoder reranking and query-decomposition pipelines.

Acknowledgments

We would like to express our deepest gratitude to our supervisor, **Dr. Nguyen Kiem Hieu**, for his continuous guidance, invaluable feedback, and unwavering encouragement throughout this project. His profound insights into both natural language processing and the practical complexities of developing legal AI systems fundamentally shaped the direction of this work.

We are also deeply thankful to our teammate, **Vu Minh Hieu**, for his enthusiastic support, collaborative spirit, and meaningful contributions to this project.

Finally, we would like to acknowledge the hard work, dedication, and shared perseverance of our entire research team. This project stands as a testament to our collective effort and commitment.

Hanoi, 2026
Nguyen Nhat Minh
Vu Duc Duy

Bibliography

[1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[2] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543, Association for Computational Linguistics, 2014.

[3] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2227–2237, Association for Computational Linguistics, 2018.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186, Association for Computational Linguistics, 2019.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, Association for Computational Linguistics, 2019.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, Curran Associates, Inc., 2020.

[10] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.

[11] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, "How does NLP benefit legal system: A summary of legal artificial intelligence," *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5218–5230, 2020.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[13] H. Bui, "Vietnamese spelling and diacritic correction model (v2)." `https://huggingface.co/bmd1905/vietnamese-correction-v2`, 2024.

[14] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.